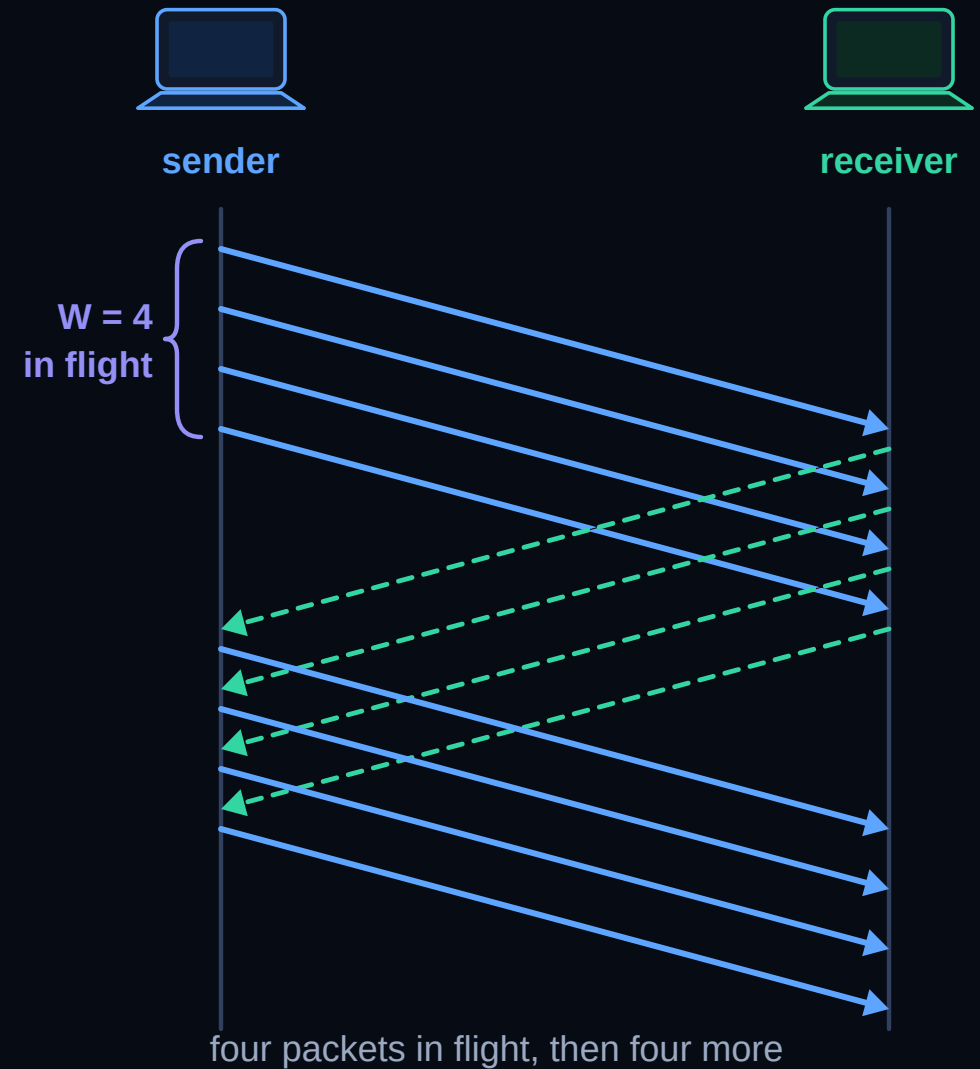


Pipelining and Sliding Windows

Many packets in flight

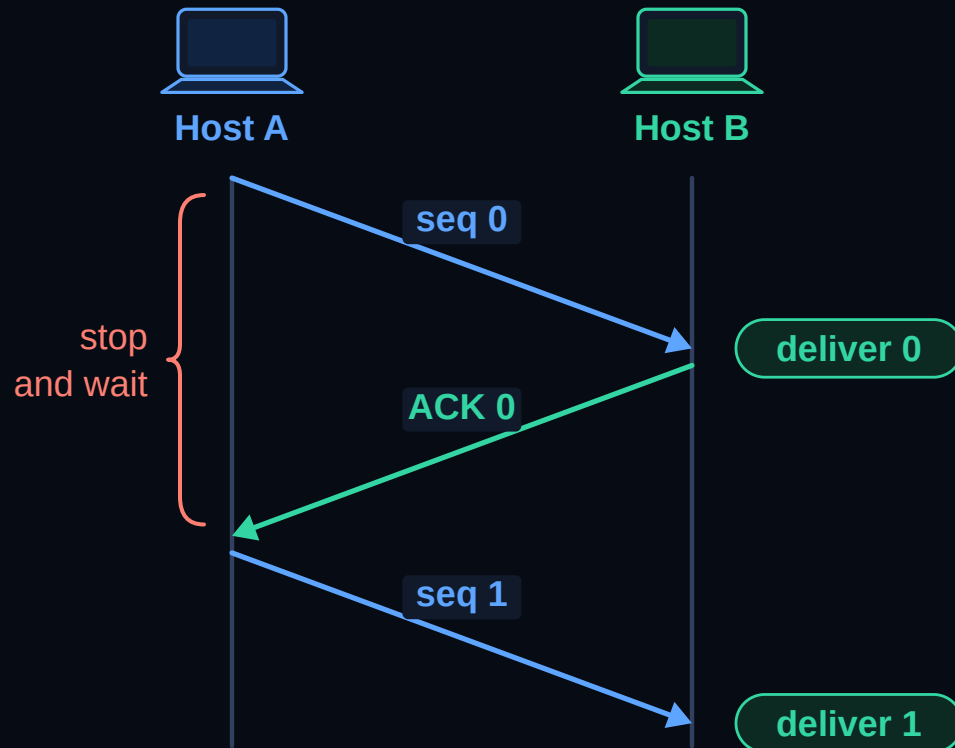
Part 2 built a protocol that survives bit errors, loss and delay, one packet at a time. This lecture measures what one-at-a-time costs, puts a window of packets in flight, and decides what to resend when one of them is lost.

Yaxiong Xie



- ——— Where Part 2 stopped

The protocol so far has a name: stop-and-wait



stop-and-wait

the protocol Part 2 built

**send one packet · stop · wait for its ACK
only then send the next one**

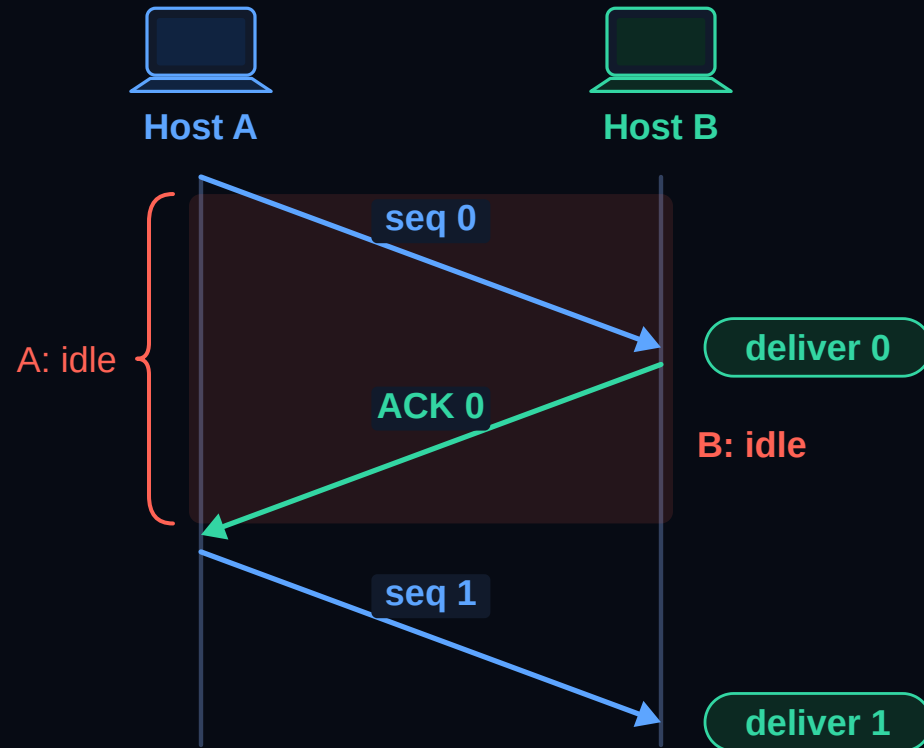
one packet in flight at any moment

It survives bit errors, loss and delay:
every packet arrives, once, in order.

Correct. A sends one packet and does nothing until its ACK is back.

- ——— Where Part 2 stopped

Is there a problem with stop-and-wait?



Is there a problem with stop-and-wait?

It is correct. Is it also fast?

Between seq 0 and seq 1, one round trip passes. A is idle. B is idle. The link carries nothing.

How much time is lost this way, and what does it cost?

Part 1 answers this with numbers: how slow stop-and-wait is, and why.

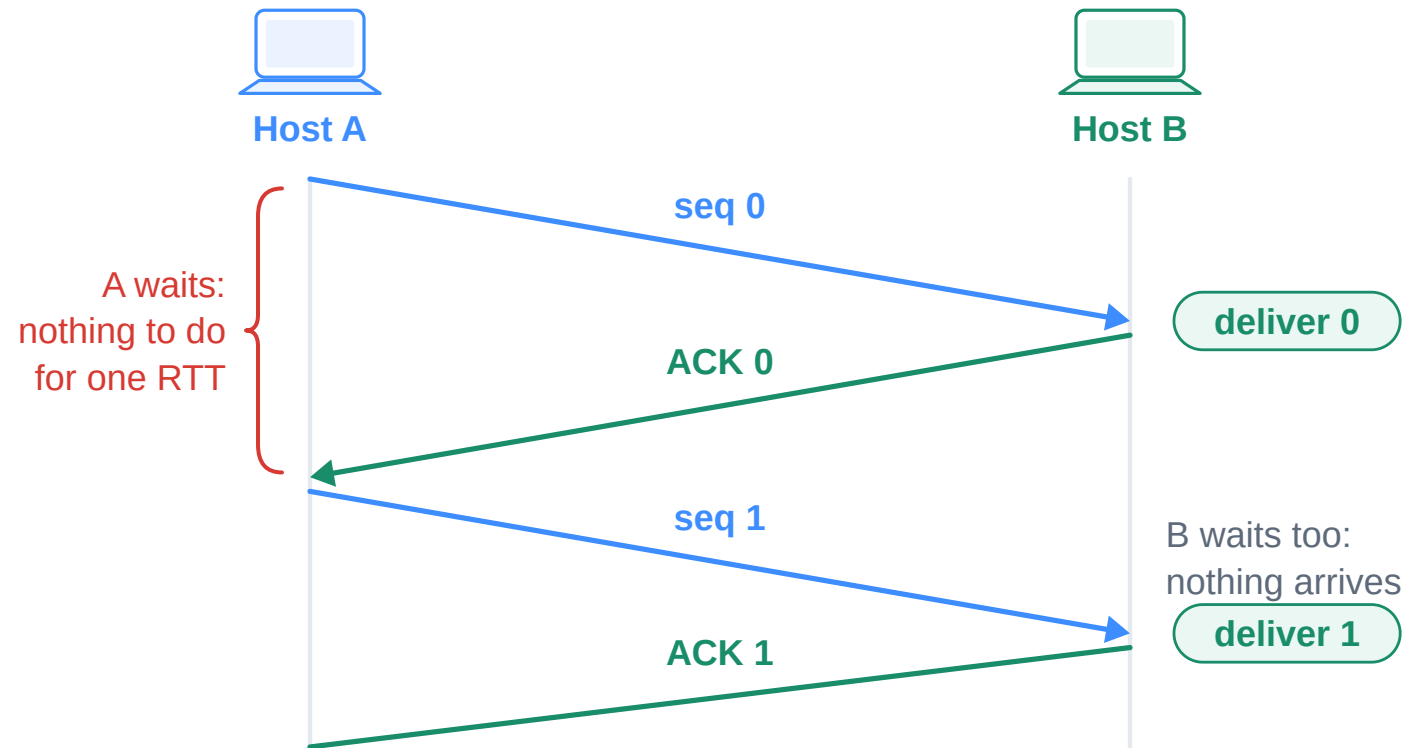
Correct, but slow: what does waiting cost?

01

Pipelining

The protocol so far sends one packet and waits for its ACK. This part names that design, measures what the waiting costs, and removes it.

What does A do while it waits for the ACK?



Nothing. Stop-and-wait: one packet per round trip, and A is idle for most of it.

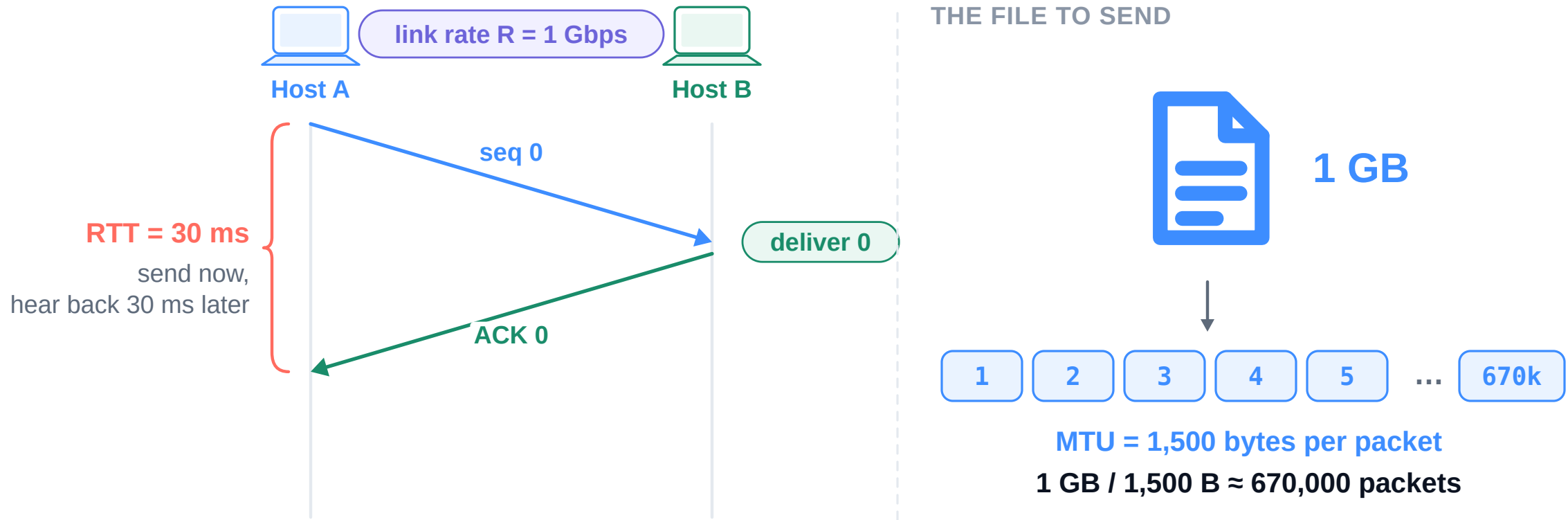
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

The running example: a 1 GB file over a 1 Gbps link



R = 1 Gbps, RTT = 30 ms, 670,000 packets of 1,500 bytes.

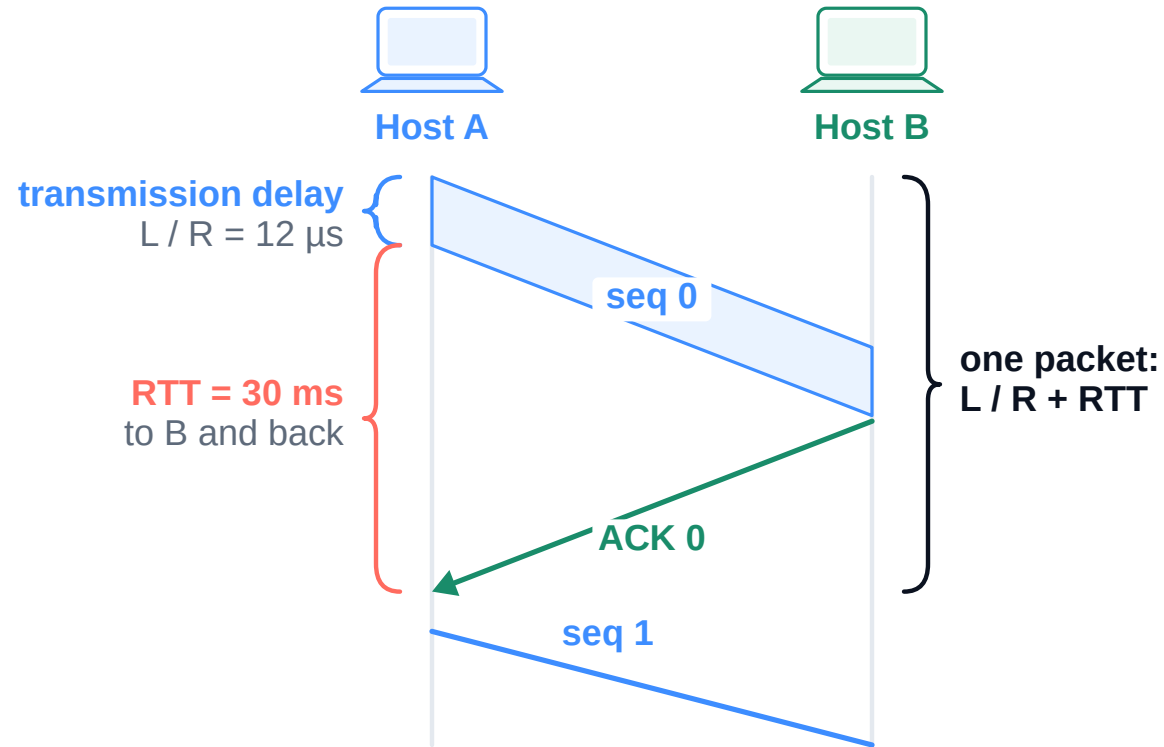
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

How long does one packet take with stop-and-wait?



not to scale: $12 \mu s$ is 2,500 times shorter than 30 ms

→ First: push the bits onto the link

$$L / R = \frac{1,500 \times 8 \text{ bit}}{10^9 \text{ bit/s}} = 12 \mu s$$

⌚ Then: to B, and the ACK back to A

the round-trip time, $RTT = 30 \text{ ms}$;
A cannot know sooner that the packet arrived

one packet = $L / R + RTT$

= $0.012 \text{ ms} + 30 \text{ ms} \approx 30 \text{ ms}$

$L / R + RTT \approx 30 \text{ ms}$ per packet. The round trip sets the pace, not the link rate.

1 Pipelining

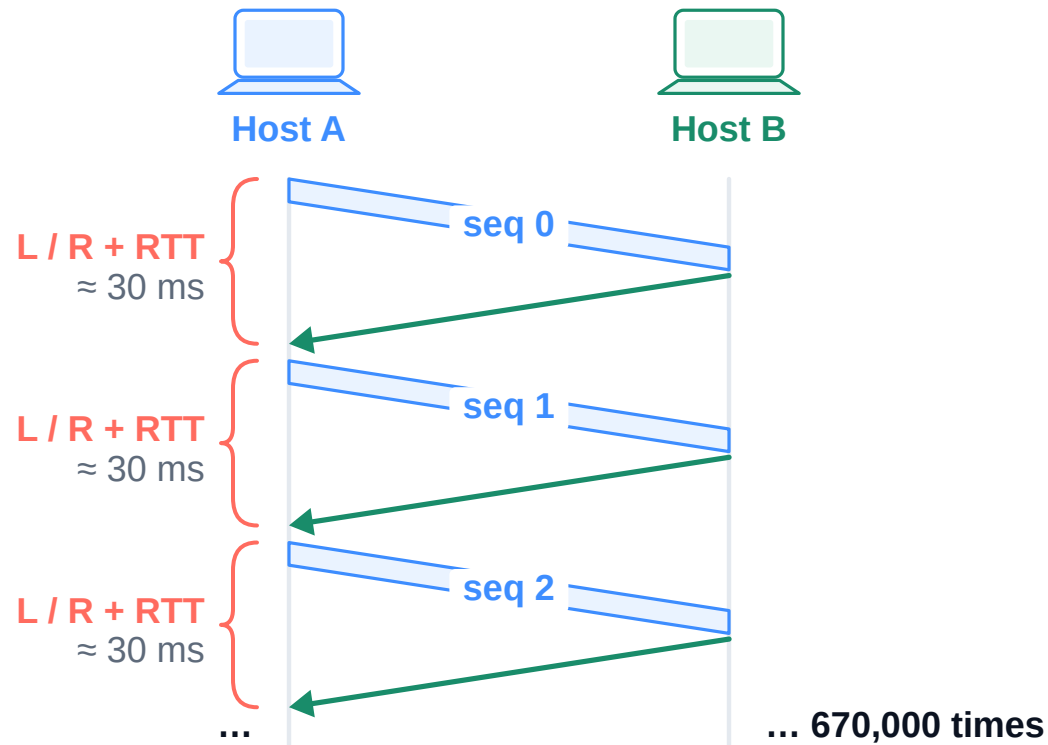
2 Sliding window

3 Go-Back-N

4 Selective repeat

● — Pipelining

How long does the 1 GB file take with stop-and-wait?



670,000 packets $\times$ 30 ms

= 20,000 s

one packet per round trip, one after the other

5.6 h

for a 1 GB file

The link alone would take 8 s. A 10 Gbps link
changes the 12 μ s, not the 30 ms: still 5.6 h.

5.6 hours, at any link rate: stop-and-wait pays one round trip per packet.

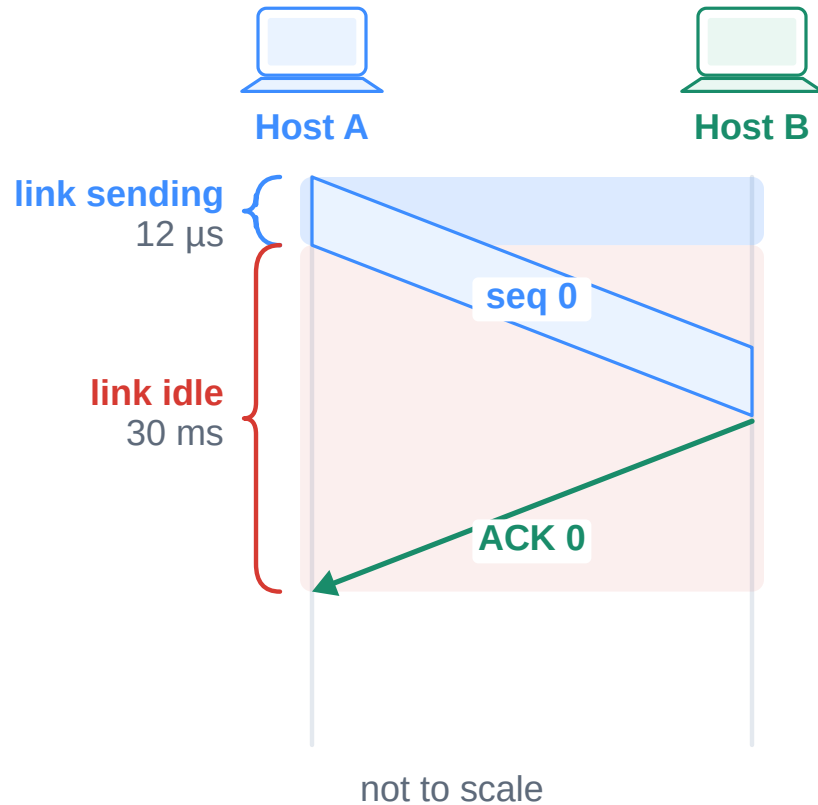
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

How much of the time is the link actually sending?



$$\text{utilization} = \frac{\text{time the link is sending}}{\text{total time}}$$

between 0 (never used) and 1 (never idle)

$$U = \frac{L / R}{L / R + \text{RTT}} = \frac{12 \mu\text{s}}{12 \mu\text{s} + 30,000 \mu\text{s}}$$
$$\approx 1 / 2,500 = 0.0004 = 0.04\%$$

Utilization: the share of time the link carries data. Here 12 μ s in 30 ms: 0.04%.

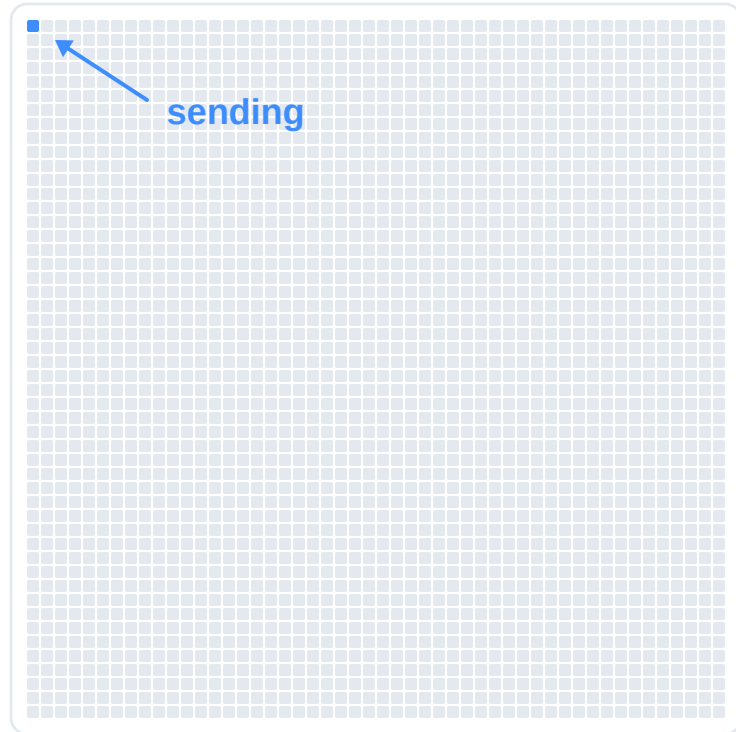
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

How low is the utilization of stop-and-wait here?



2,500 slots of 12 μ s = one round trip

0.04%

of the time, the link carries data
the other 99.96% it waits

one 12 μ s slot of sending,
then 2,499 slots of waiting for the ACK

One blue square. Stop-and-wait uses 0.04% of a 1 Gbps link with a 30 ms RTT.

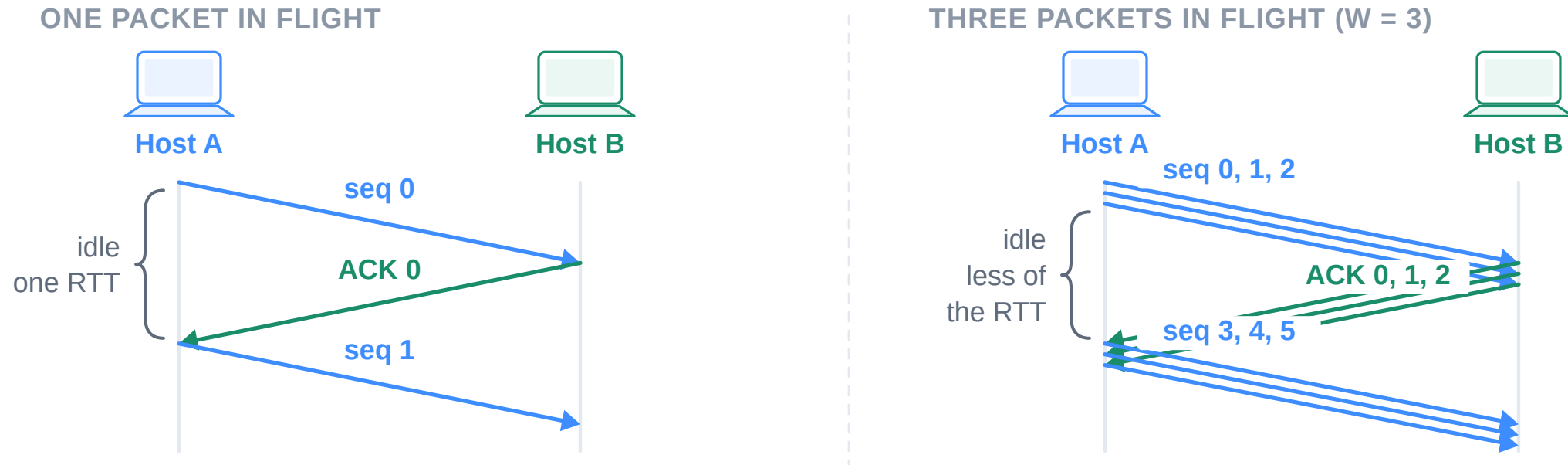
1 **Pipelining**

2 Sliding window

3 Go-Back-N

4 Selective repeat

Why not send several packets before the first ACK?



throughput $\leq W \cdot L / \text{RTT}$ — the same round trip now carries W packets instead of one.

W packets in flight multiply throughput by W , until the link is full.

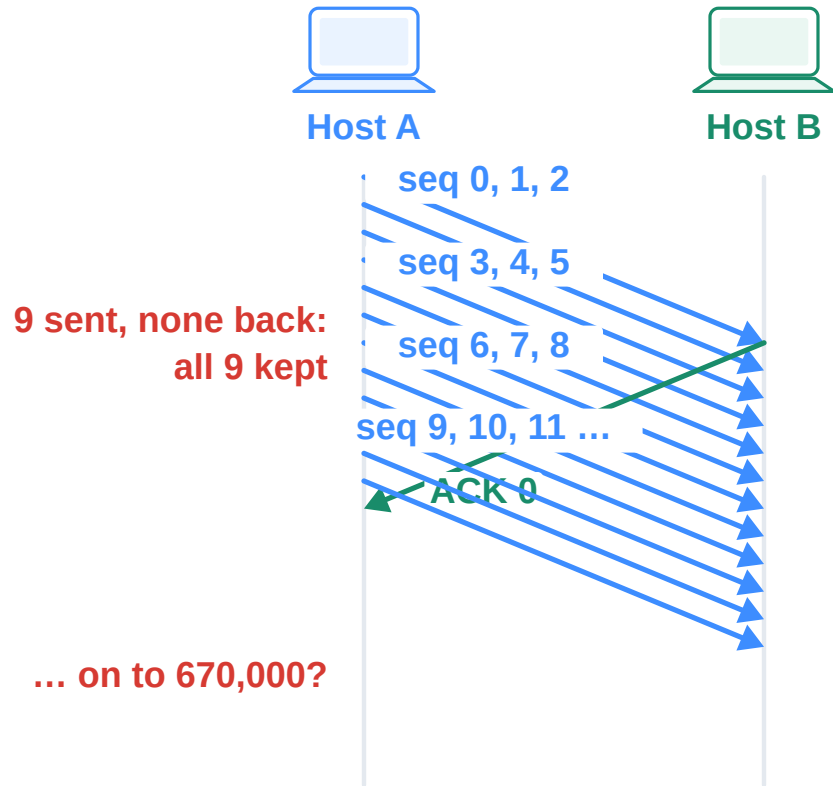
1 **Pipelining**

2 Sliding window

3 Go-Back-N

4 Selective repeat

Keep sending until the first ACK comes back?



 **A: every packet out is a copy kept**

until its ACK is back it may have to be resent;
on to 670,000 packets, 1 GB waits in memory

 **B: packets arrive faster than they are read**

B's buffer fills; what does not fit is dropped
and has to be sent again

 **The link: at most R bits per second**

once the link is full, extra packets only wait
in router queues, or are dropped there

No. Memory at A, drops at B, queues on the link: A needs a limit on how many are out.

1 **Pipelining**

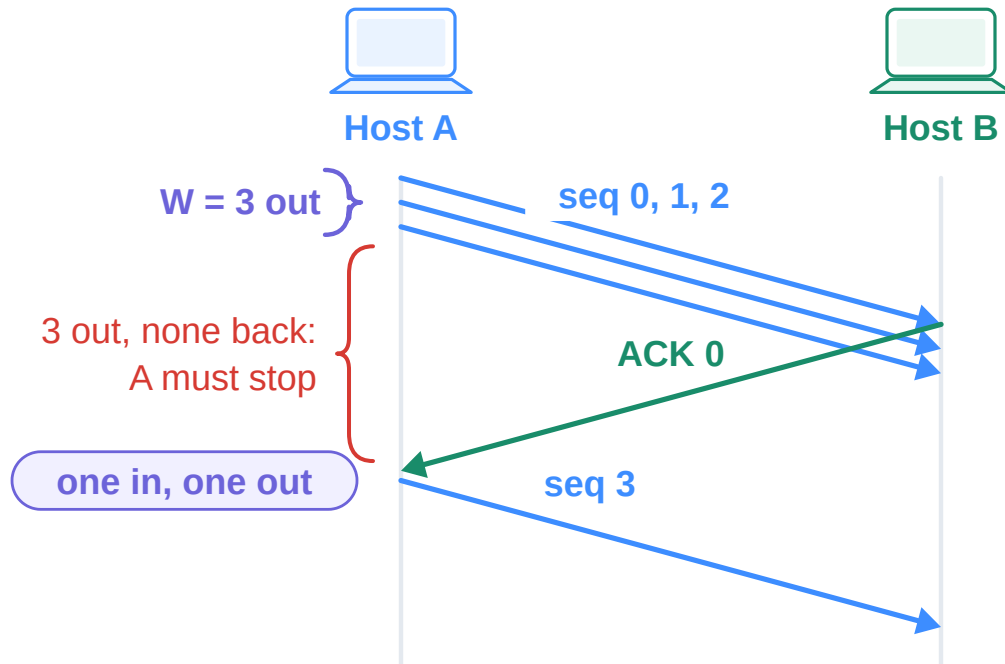
2 Sliding window

3 Go-Back-N

4 Selective repeat

A limit: at most W packets sent and not yet acknowledged

IN THE TIME DIAGRAM, $W = 3$



A stores at most W packets

every one of them may still have to be resent

B receives at most W

before A has heard back about any of them

W is the design parameter. Too small and the link idles again; how large is enough?

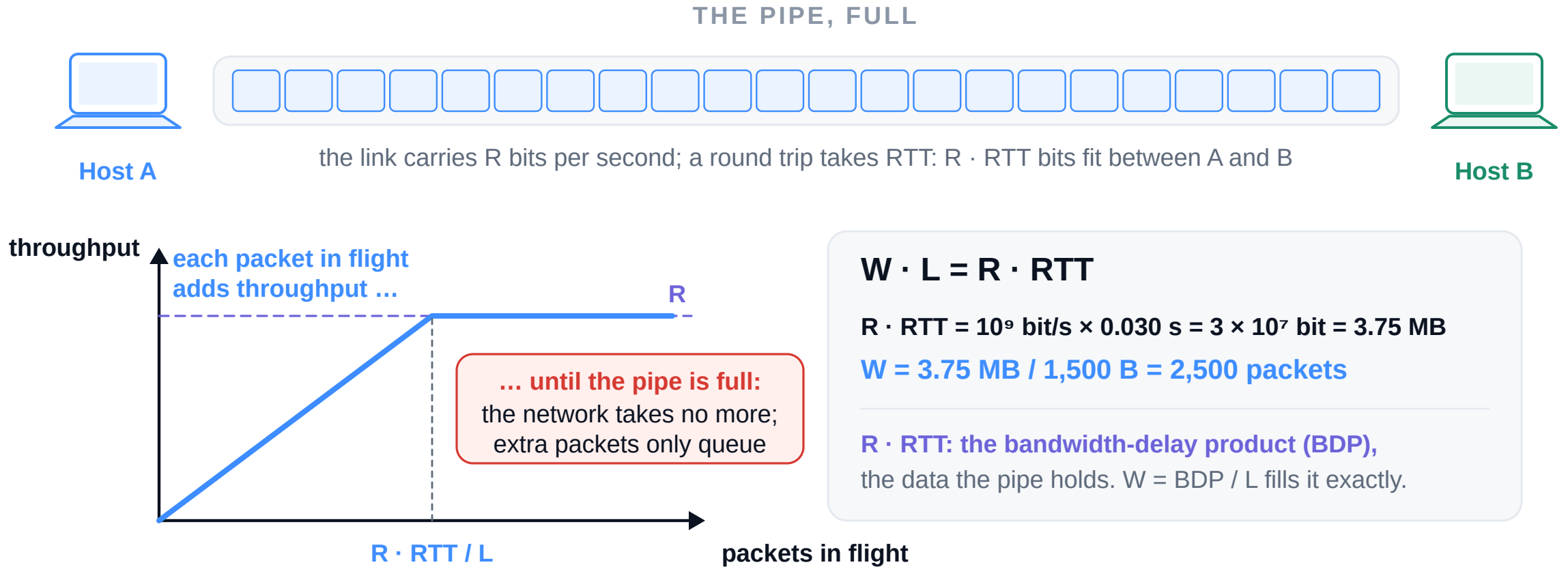
1 **Pipelining**

2 Sliding window

3 Go-Back-N

4 Selective repeat

How large should W be?



Until the pipe is full: $W \cdot L = R \cdot RTT$, the bandwidth-delay product. Here $W = 2,500$.

1 Pipelining

2 Sliding window

3 Go-Back-N

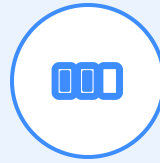
4 Selective repeat

Pipelining changes three things at once



Sequence numbers

one bit is no longer enough;
W packets in flight must
all carry different numbers



A sender buffer

every unacknowledged packet
is kept, because any of
them may have to be resent



Out-of-order arrival

packet 3 can arrive before
packet 2 does;
B must decide what to do

Part 2 shows how A keeps track of the W packets. Parts 3 and 4 decide what B does with a gap.

Each of the three needs a design decision. The rest of this lecture makes them.

1 **Pipelining**

2 Sliding window

3 Go-Back-N

4 Selective repeat

How does A keep exactly W packets in flight?

02

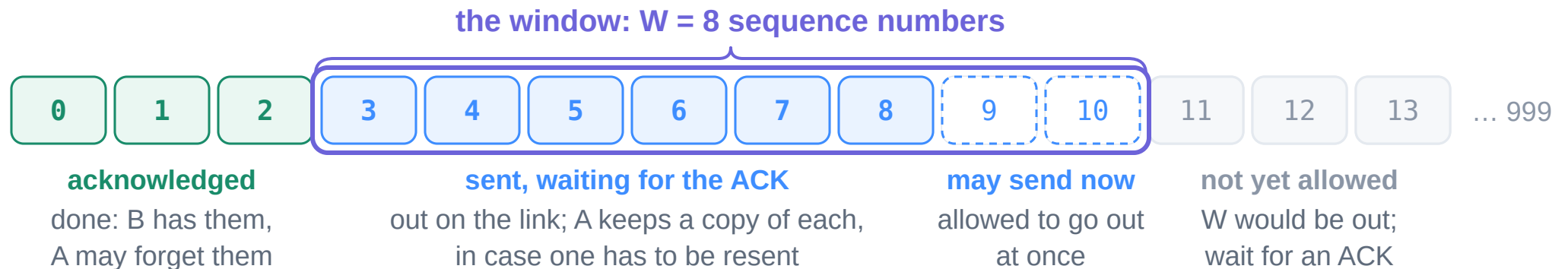
Sliding window

A must know which packets are done, which are out, which it may send now and which it may not. One picture holds all of it.

Which packets does A have to keep track of?

A FILE OF 1,000 PACKETS, ALL READY AT A

Every packet carries its sequence number, 0 ... 999. The strip shows the first 14, seen from A.



The sliding window: the W sequence numbers from the oldest unacknowledged packet on.

It holds the packets that are out (most of it) and the packets A may still send (the rest).
Packets left of it are done; packets right of it must wait until the window moves.

Four kinds. The window: W numbers, from the oldest unacknowledged packet on.

1 **Pipelining**

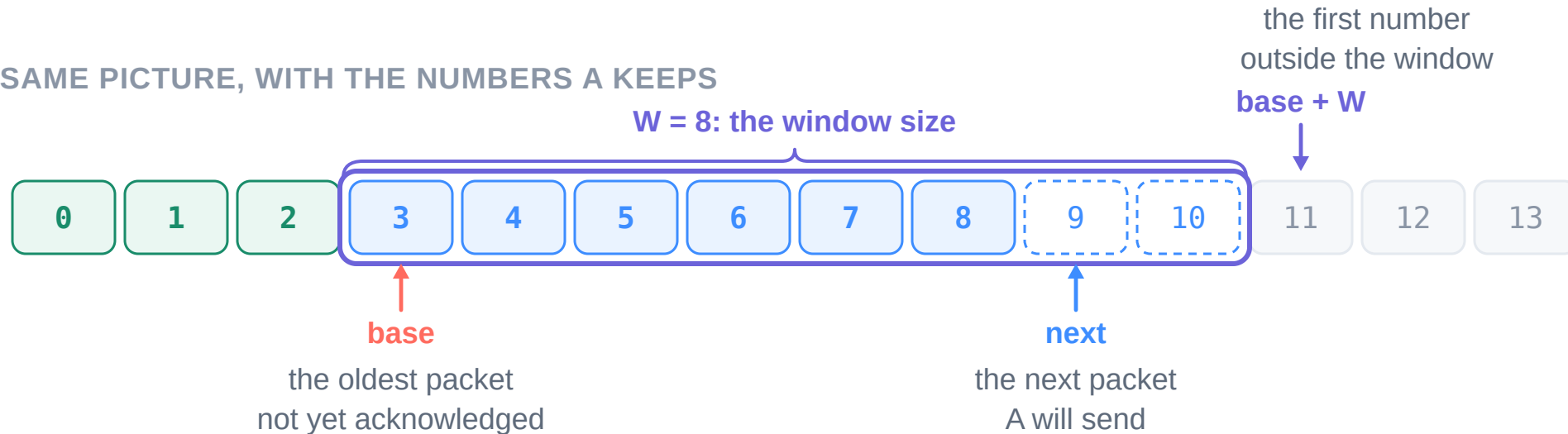
2 **Sliding window**

3 Go-Back-N

4 Selective repeat

The numbers that run the window: base, next and W

THE SAME PICTURE, WITH THE NUMBERS A KEEPS



window = base ... base + W - 1

sent, waiting = base ... next - 1

may send now = next ... base + W - 1

A may send $\Leftrightarrow$ next < base + W

A keeps three numbers

base and next change as packets go out and ACKs come in; W is fixed by the design. Plus the copies of packets base ... next - 1.

base: oldest unacknowledged. next: first unsent. W: the size. Send while next < base + W.

1 Pipelining

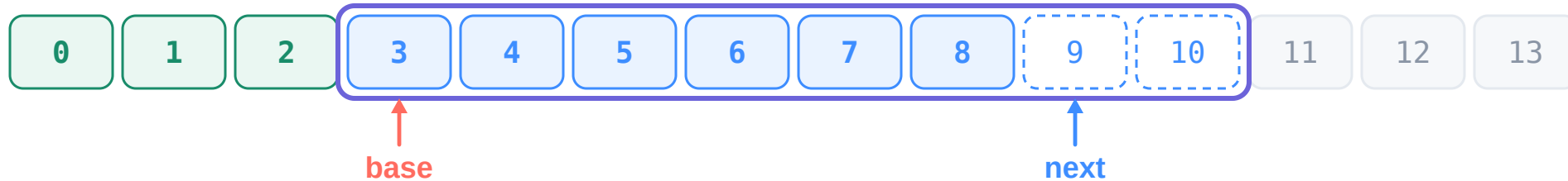
2 Sliding window

3 Go-Back-N

4 Selective repeat

What happens when A sends a packet?

$W = 8$. $BASE = 3$, $NEXT = 9$: TWO PACKETS MAY STILL GO OUT



to the link

send packet next: keep a copy, start its timer, $next = next + 1$

if $next = base + W$ afterwards, the window is full: A may not send again until an ACK arrives
the window itself does not move when A sends

Each send moves next right by one. When next reaches $base + W$, A must wait for an ACK.

1 Pipelining

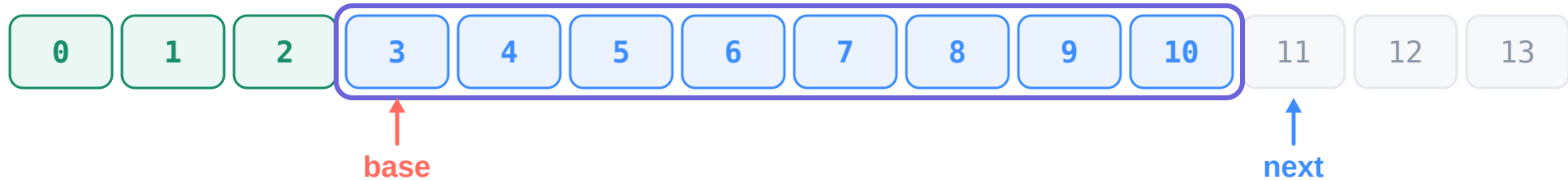
2 Sliding window

3 Go-Back-N

4 Selective repeat

What happens when the ACK for base arrives?

W = 8. BASE = 3, NEXT = 11: THE WINDOW IS FULL, ACK 3 ARRIVES



ACK 3

ACK base arrives: stop its timer, drop the copy, $\text{base} = \text{base} + 1$

the window moves with base: one new number, $\text{base} + W - 1$, becomes usable
 $\text{next} < \text{base} + W$ again, so A may send one more packet

base moves right by one and the window moves with it. One more packet may go out.

1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

Why is it called a sliding window?

A'S SEQUENCE NUMBERS; A SMALL WINDOW HERE, $W = 4$, SO THE SLIDE IS EASY TO FOLLOW



Each ACK for base moves the window one slot to the right and lets one new packet out.

A never has more than W packets out, and never fewer while it has data to send.

The window slides over the sequence numbers as ACKs come back.

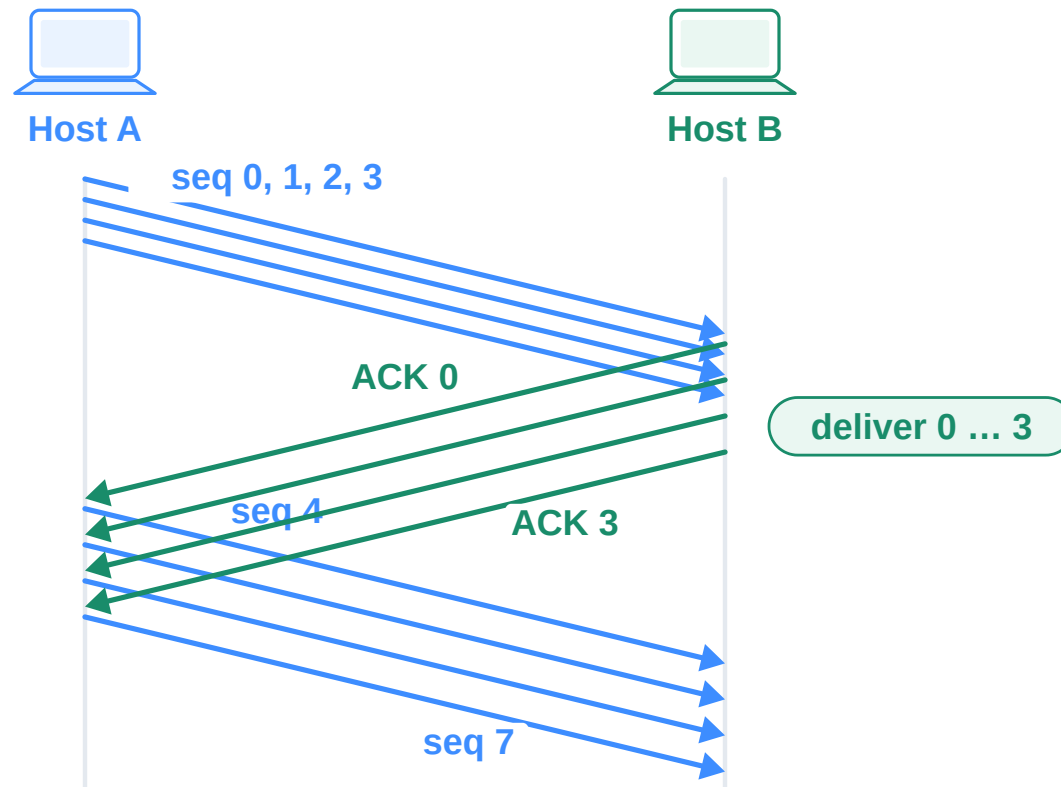
1 **Pipelining**

2 **Sliding window**

3 Go-Back-N

4 Selective repeat

In the time diagram: one ACK in, one packet out



A'S WINDOW AFTER EACH STEP



A sends 7
four out again

Steady state

ACK k arrives, packet $k + W$ leaves.
Exactly W packets are unacknowledged
at every moment. The link never idles.

With $W = R \cdot \text{RTT} / L$, the ACK for packet k arrives just as packet $k + W$ is needed.

1 **Pipelining**

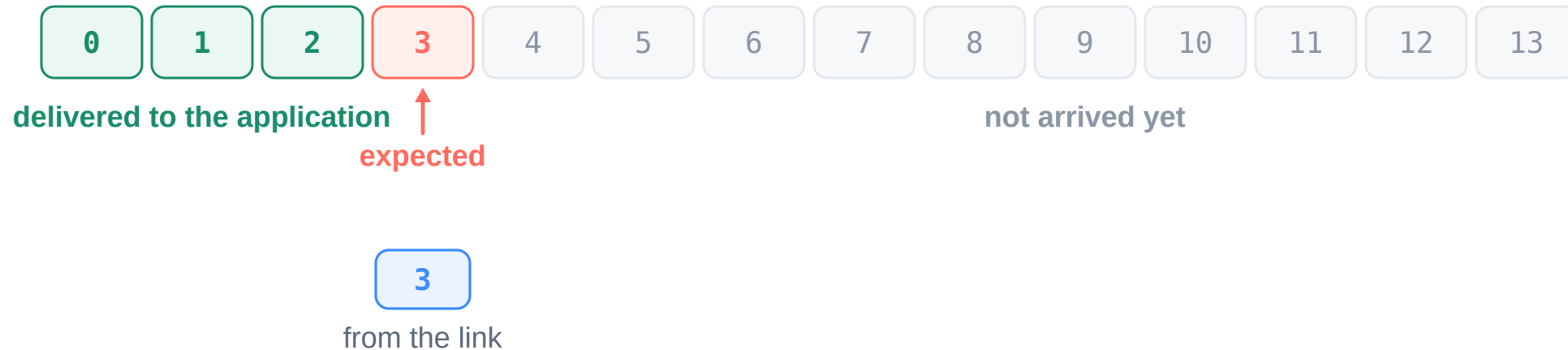
2 **Sliding window**

3 Go-Back-N

4 Selective repeat

What does B keep track of?

THE SEQUENCE NUMBERS, SEEN FROM B



packet expected arrives: deliver it, send ACK, $\text{expected} = \text{expected} + 1$

B keeps one number. With no loss and no reordering, only expected ever arrives.

❓ And if packet 4 arrives while B still expects 3? Parts 3 and 4.

One number: expected. B delivers the expected packet, acknowledges it and moves on.

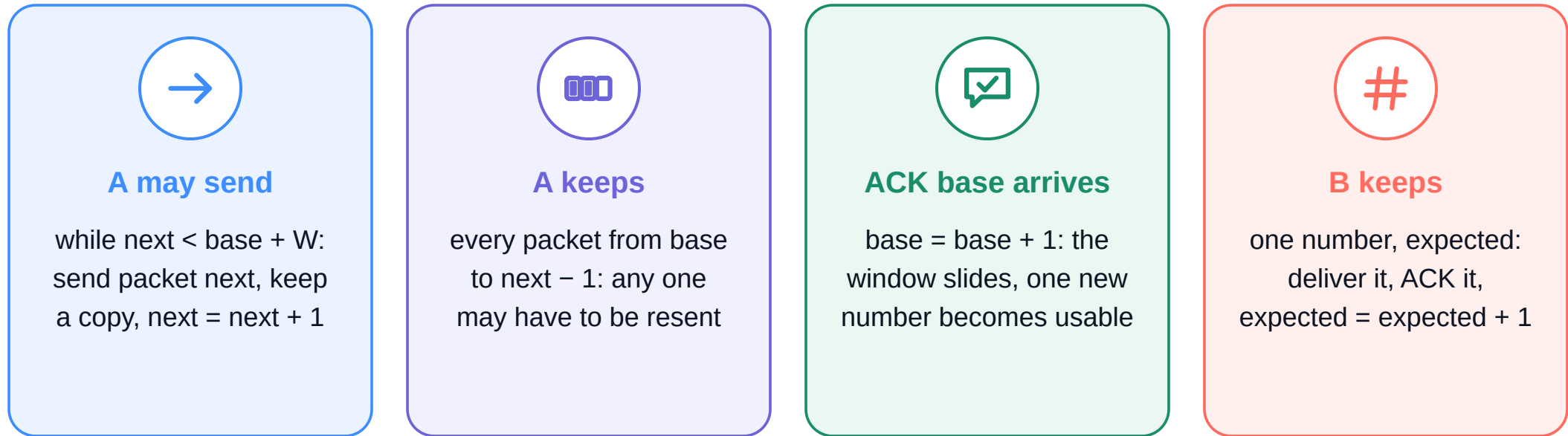
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

The sliding window in four rules



Four rules, two numbers at A, one number at B. Nothing yet says what happens when a packet is lost.

base, next and expected: three numbers run the whole window.

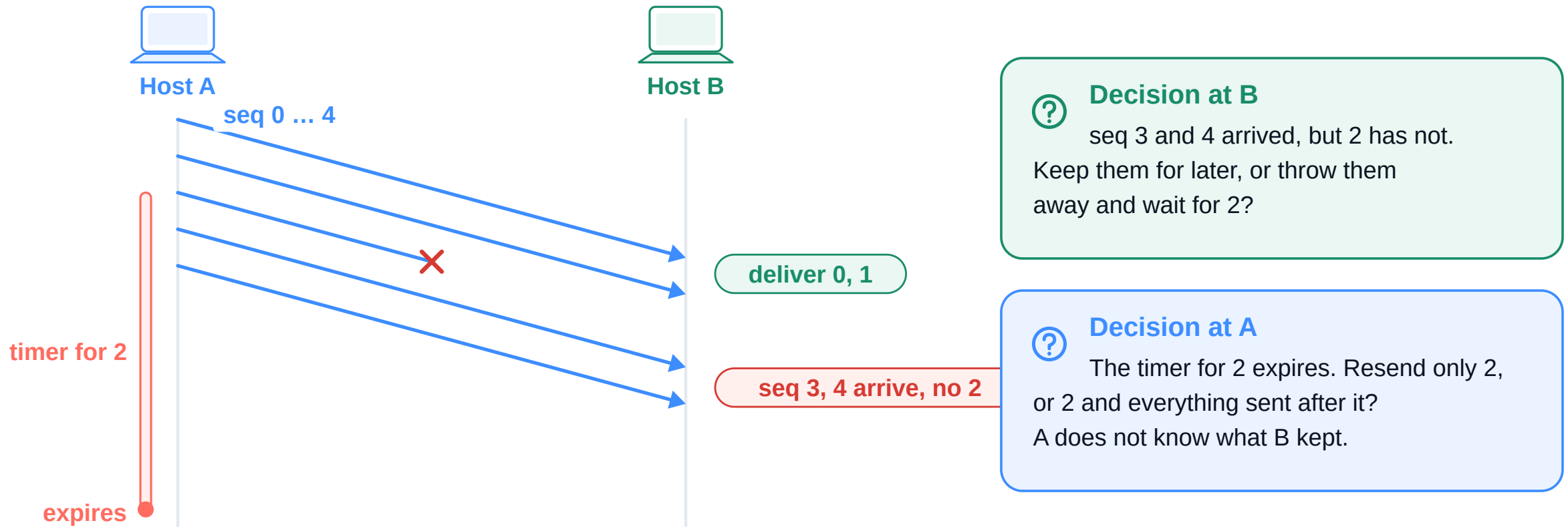
1 **Pipelining**

2 **Sliding window**

3 Go-Back-N

4 Selective repeat

What if one packet inside the window is lost?



The two decisions have to agree. Each pair of answers is a different protocol.

1 **Pipelining**

2 **Sliding window**

3 Go-Back-N

4 Selective repeat

What if B keeps nothing out of order?

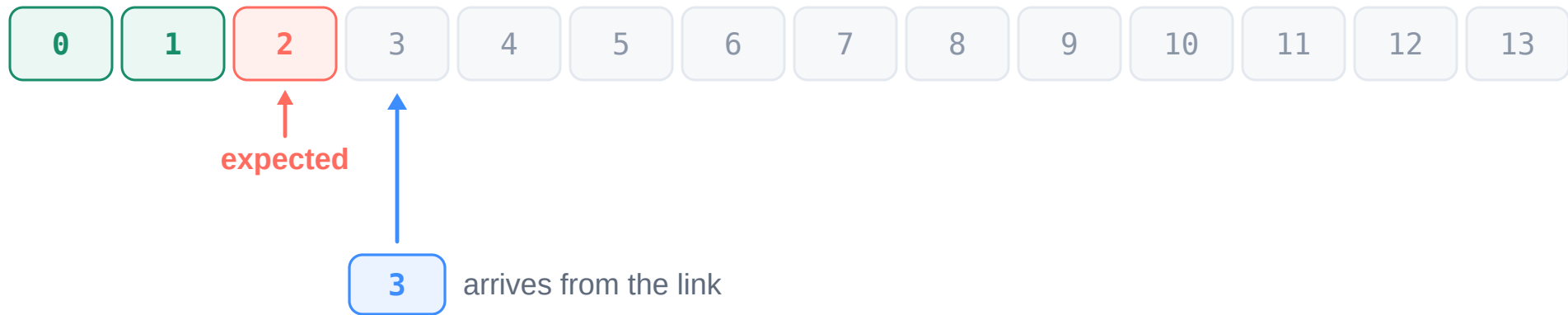
03

Go-Back-N

The simplest answer at B: a packet that is not the expected one is thrown away. This part follows that answer to its cost.

What does B do with packet 3 while it waits for 2?

SEEN FROM B: 0 AND 1 DELIVERED, 2 IS EXPECTED



? Packet 3 arrives, but B is waiting for packet 2. What should B do with 3?

3 is not the expected packet. B has to decide: keep it, or throw it away.

1 Pipelining

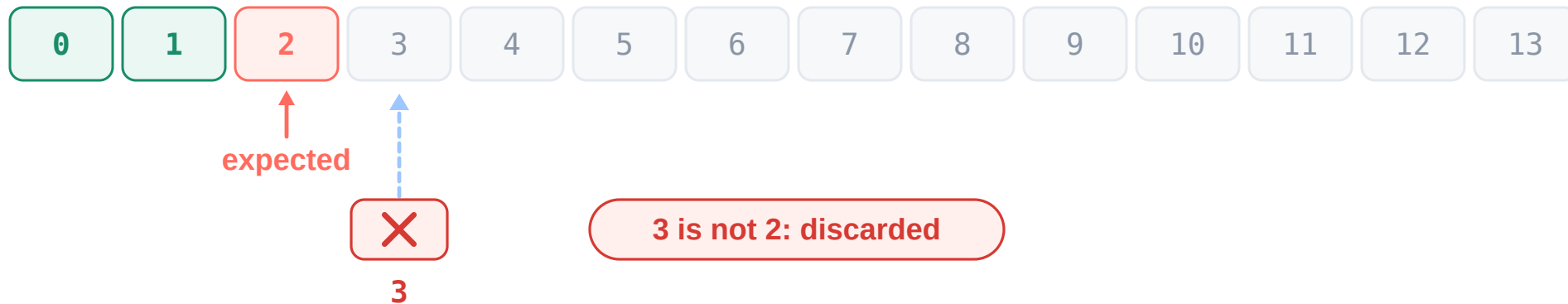
2 Sliding window

3 Go-Back-N

4 Selective repeat

Go-Back-N: B throws packet 3 away

SEEN FROM B: 0 AND 1 DELIVERED, 2 IS EXPECTED



a packet that is not expected arrives: discard it

the simplest possible receiver: no buffer for packets out of order, nothing to remember about them

Out of order means discarded. B keeps nothing but the expected number.

1 Pipelining

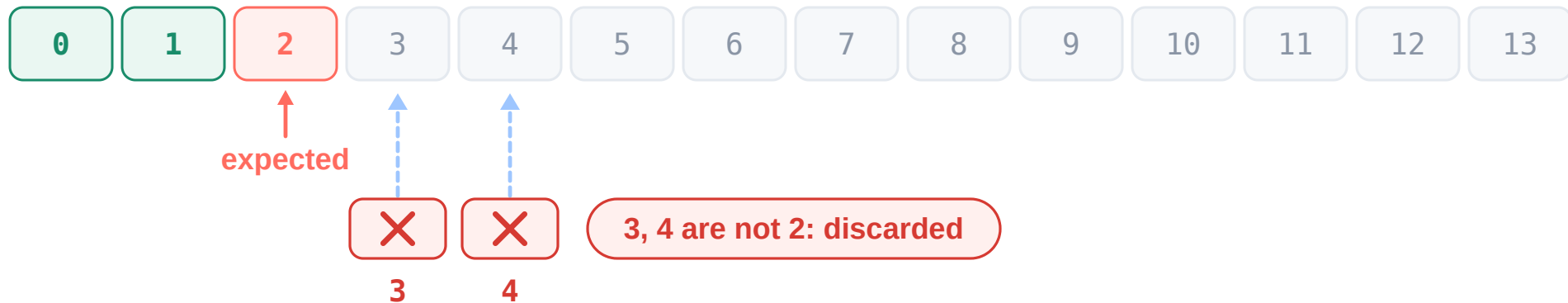
2 Sliding window

3 Go-Back-N

4 Selective repeat

expected stays 2: packet 4 is discarded the same way

SEEN FROM B: 0 AND 1 DELIVERED, 2 IS EXPECTED



expected stays 2 until packet 2 itself arrives

packet 4 gets the same treatment; so would 5, 6, ... — everything until 2 is resent

B throws 3 and 4 away. expected stays 2; B keeps nothing out of order.

1 Pipelining

2 Sliding window

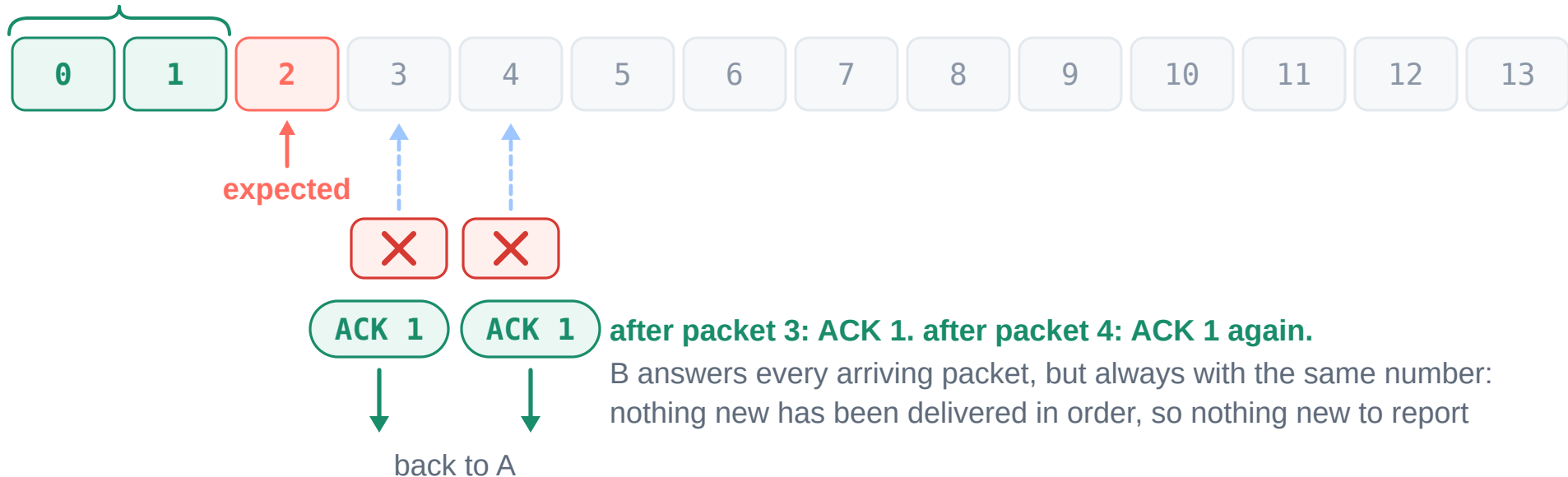
3 Go-Back-N

4 Selective repeat

What should B tell A after it discards packets 3 and 4?

SEEN FROM B: 3 AND 4 ARRIVED AND WERE DISCARDED; 2 IS STILL EXPECTED

ACK 1 = "I have every packet up to 1" — the last one delivered in order



cumulative acknowledgement: ACK n means every packet up to and including n has arrived

ACK 1, and ACK 1 again. A cumulative ACK n says: everything up to n is here.

1 Pipelining

2 Sliding window

3 Go-Back-N

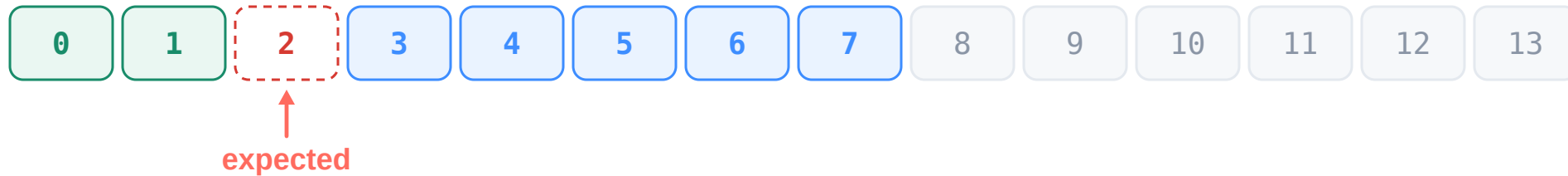
4 Selective repeat

A sends the whole window: 0 ... 7. Packet 2 is lost on the way

A'S WINDOW, W = 8



B: WHAT IT HOLDS



A sends the whole window without waiting. On the way, packet 2 is lost.

B delivers 0 and 1 and waits for 2. Then 3 ... 7 arrive, in good shape, but out of order.

Seven packets reach B. Only 0 and 1 are the ones B expected.

1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

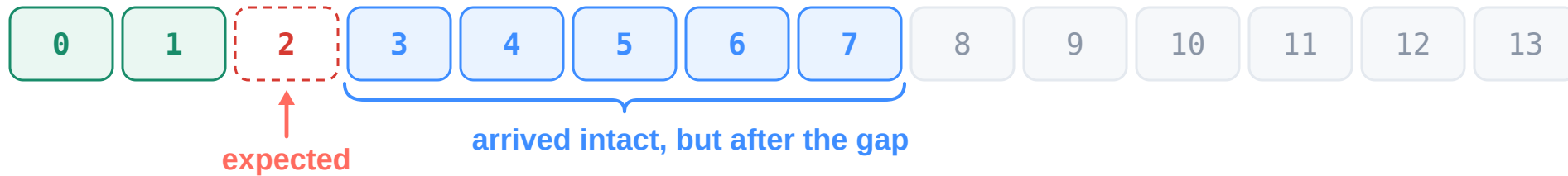
A sent 0 ... 7 and packet 2 was lost. What does B do?

A'S WINDOW, W = 8



✗
lost

B: WHAT IT HOLDS



B delivers 0 and 1. Packet 2 never arrives. 3 ... 7 arrive in good shape, but 2 is still missing.
B keeps nothing out of order: it will answer each of them, and throw it away. expected stays 2.

B delivers 0 and 1. For 3 ... 7 it has an answer to send, and nothing to keep.

1 Pipelining

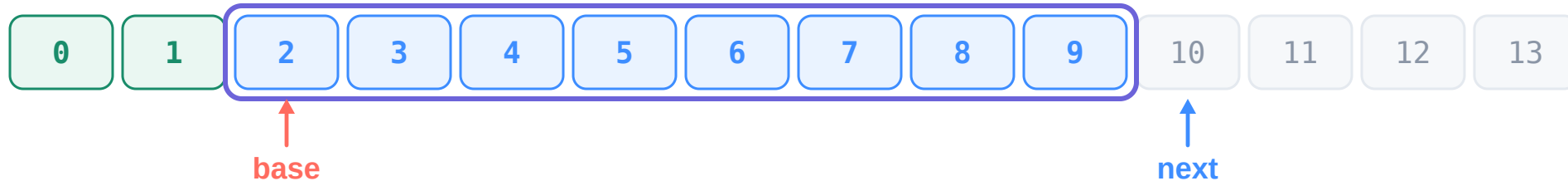
2 Sliding window

3 Go-Back-N

4 Selective repeat

B acknowledges 0 and 1: the window slides, A sends 8 and 9

A'S WINDOW, W = 8



B: WHAT IT HOLDS



ACK 0 and ACK 1 arrive: base moves to 2, the window slides to 2 ... 9, and A sends 8 and 9 at once.
8 and 9 reach B after the gap, like 3 ... 7. Now A needs ACK 2 to move on.

Two ACKs, two more packets out. Then the window waits for ACK 2.

1 Pipelining

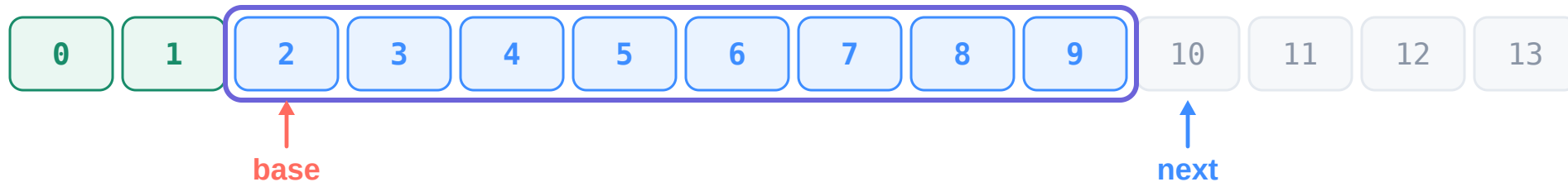
2 Sliding window

3 Go-Back-N

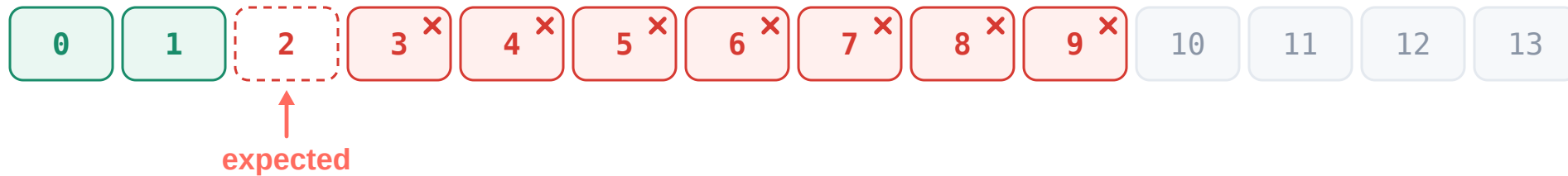
4 Selective repeat

For every packet after the gap: ACK 1, then throw it away

A'S WINDOW, $W = 8$



B: WHAT IT HOLDS



For each of 3 ... 9, B does two things: send ACK 1 (the last packet delivered in order), then discard it. The answer never changes, because nothing new is delivered in order: seven packets, seven ACK 1s, seven discards.

Seven ACKs, all saying 1, and seven good packets discarded. base stays at 2.

1 Pipelining

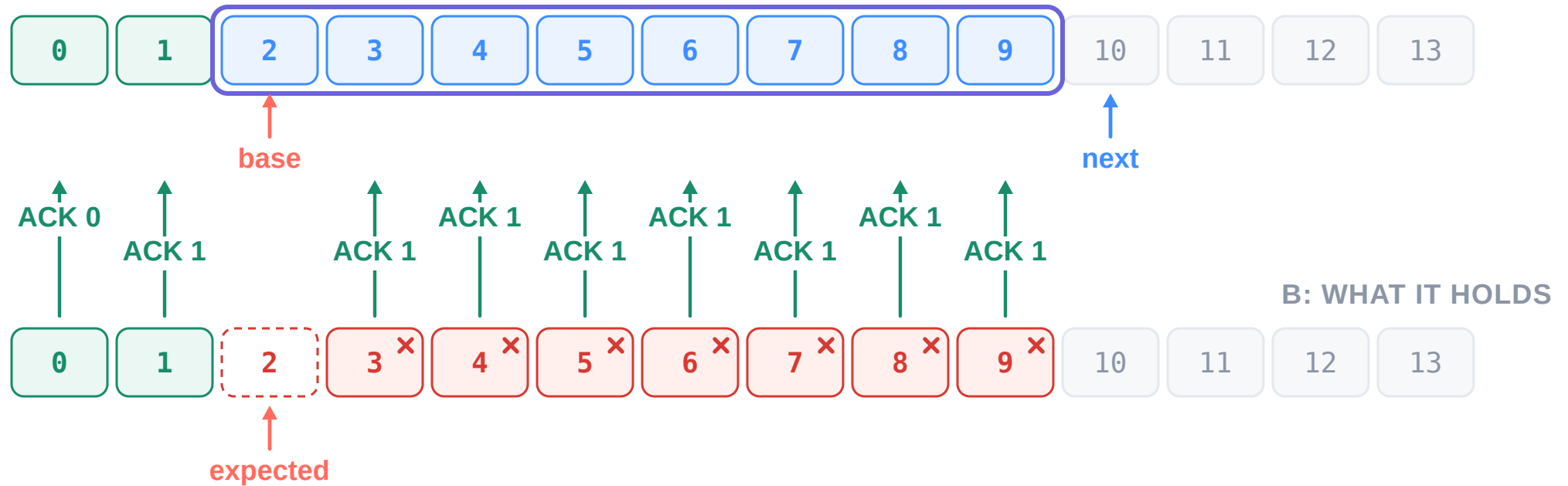
2 Sliding window

3 Go-Back-N

4 Selective repeat

What does A hear back?

A'S WINDOW, $W = 8$



ACK 0 and ACK 1 moved base to 2. Every ACK after that says 1: base does not move.
 B has thrown away 3, 4, 5, 6, 7, 8 and 9. A is stuck at 2.

ACK 0, ACK 1 — then ACK 1 seven times. base = 2, and it stays there.

1 Pipelining

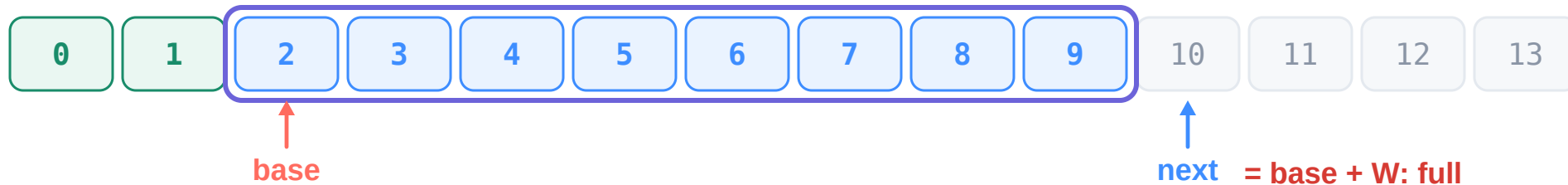
2 Sliding window

3 Go-Back-N

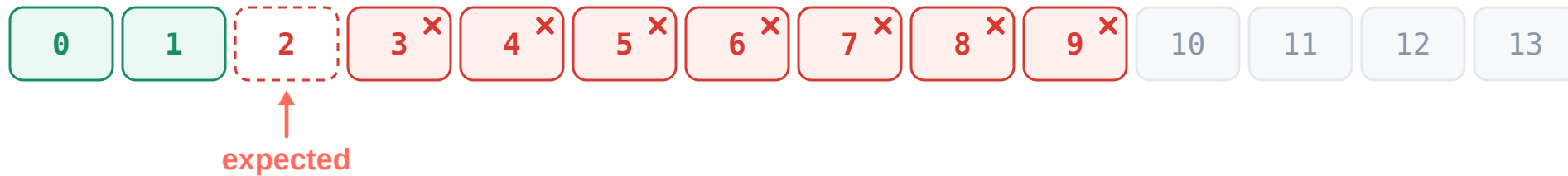
4 Selective repeat

The window is full and base does not move: A is stuck

A'S WINDOW, $W = 8$  timer for 2, running



B: WHAT IT HOLDS



Eight packets out, none of them acknowledged. $\text{next} = \text{base} + W$, so A may not send anything new.
A keeps a timer for base, packet 2. The only thing A can do now is wait for it to expire.

Eight packets out, nothing acknowledged. A sets a timer on 2 and waits.

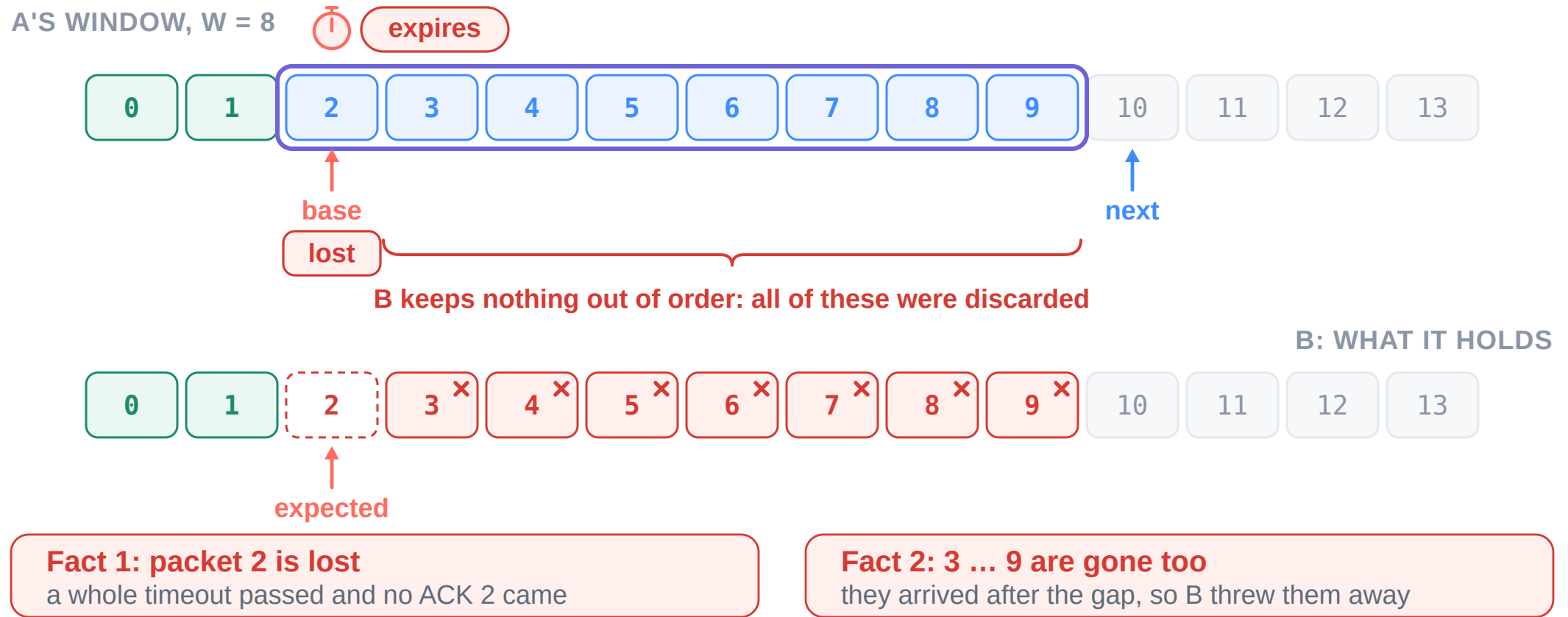
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

The timer for 2 expires. What does A know now?



Packet 2 is lost, and everything A sent after it was thrown away at B.

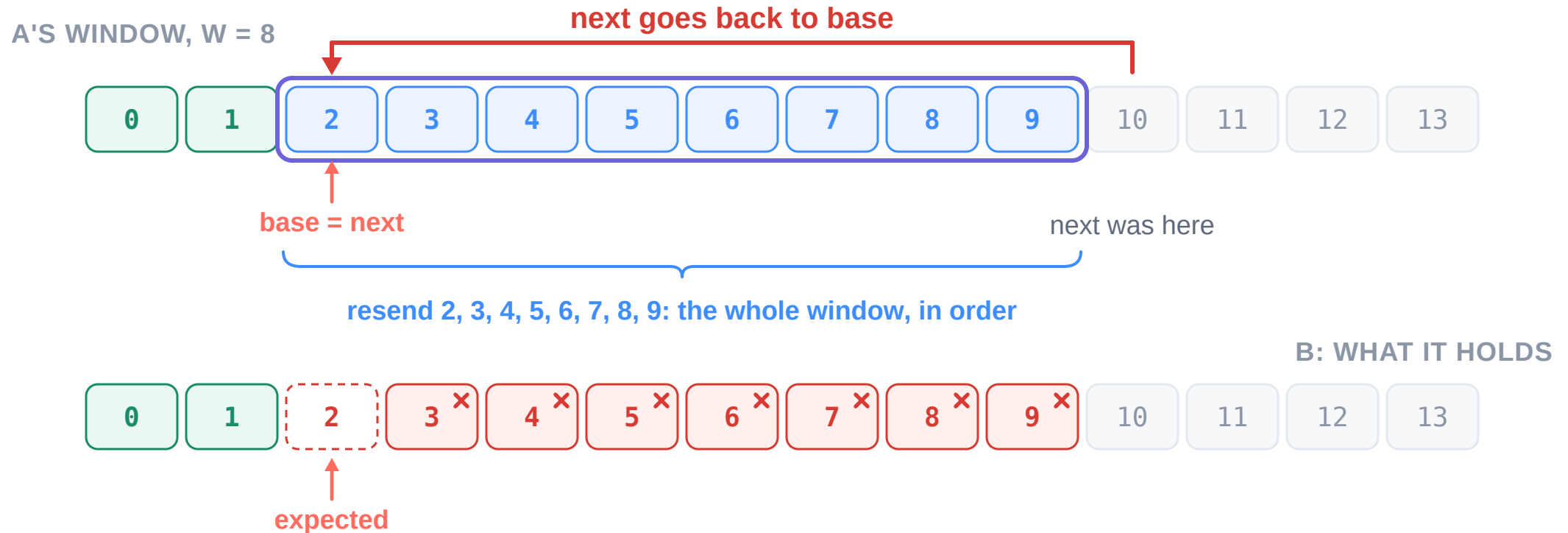
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

A goes back to base and resends the whole window



A sets $\text{next} = \text{base}$ and sends the window again: $N = W$ packets, of which only one was lost.

This is the name. Go-Back-N: the sender goes back N packets and starts over from there.

$\text{next} = \text{base}$; resend 2 ... 9. Go back $N = W$ packets: that is where the name comes from.

1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

A resends 2 ... 9. B delivers each one and acknowledges it

A'S WINDOW, W = 8



B: WHAT IT HOLDS



A sends 2, 3, 4 ... again. Each one is now the packet B expects: delivered and acknowledged.
Every ACK moves base by one. When ACK 9 arrives, the window has slid to 10 ... 17.

Eight resends, eight ACKs, and the window is moving again.

1 Pipelining

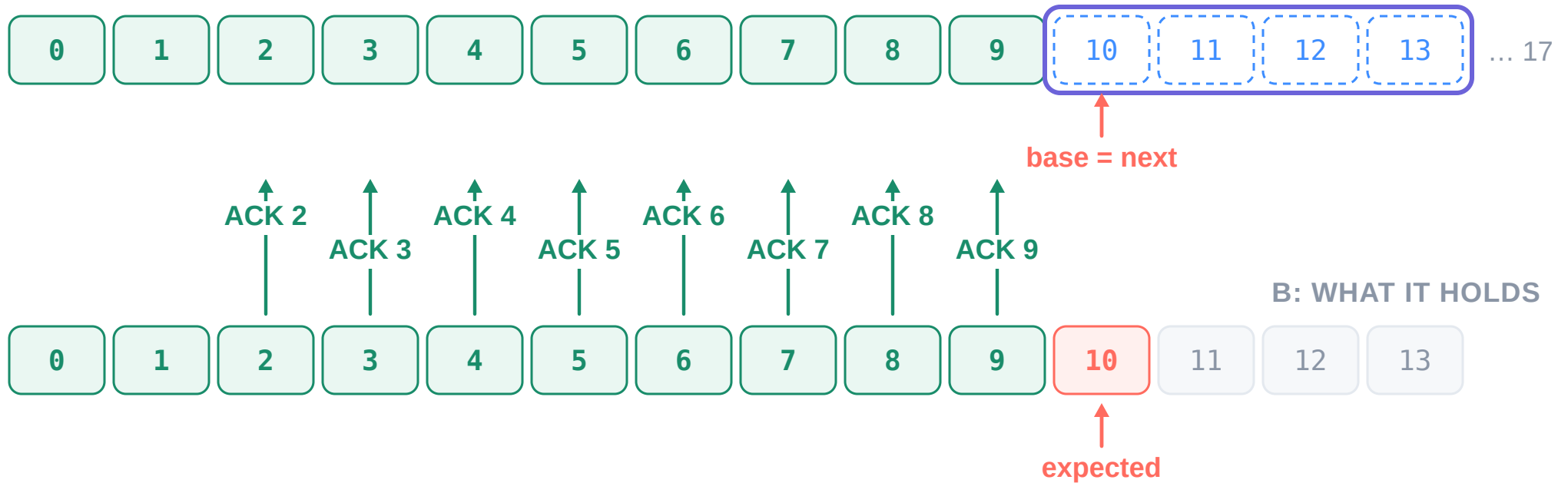
2 Sliding window

3 Go-Back-N

4 Selective repeat

The resent window arrives in order: B delivers 2 ... 9

A'S WINDOW, W = 8



Recovered. The price of one lost packet: one timeout, W retransmissions.

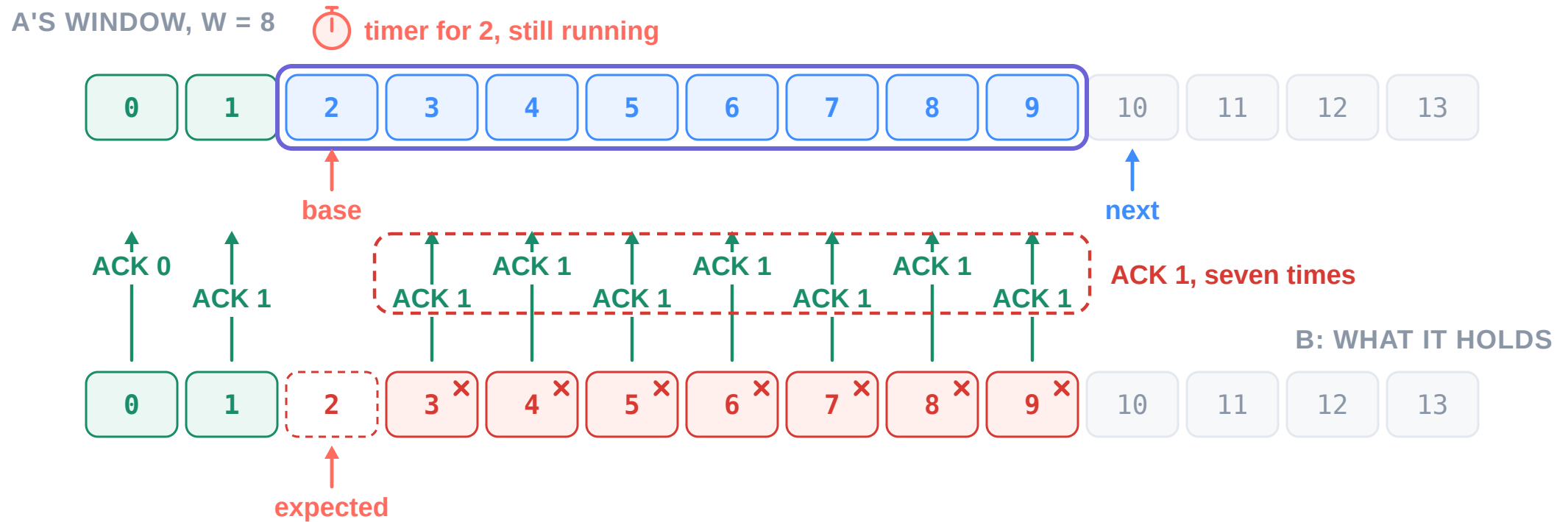
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

Did A really have to wait for the timer?



Every repeated ACK 1 says: another packet reached B, and 2 is still missing.

One duplicate could be a packet that arrived out of order. Several in a row can only mean that 2 is lost.

Repeated ACK 1s are a signal: 2 is lost. A can act before the timer expires.

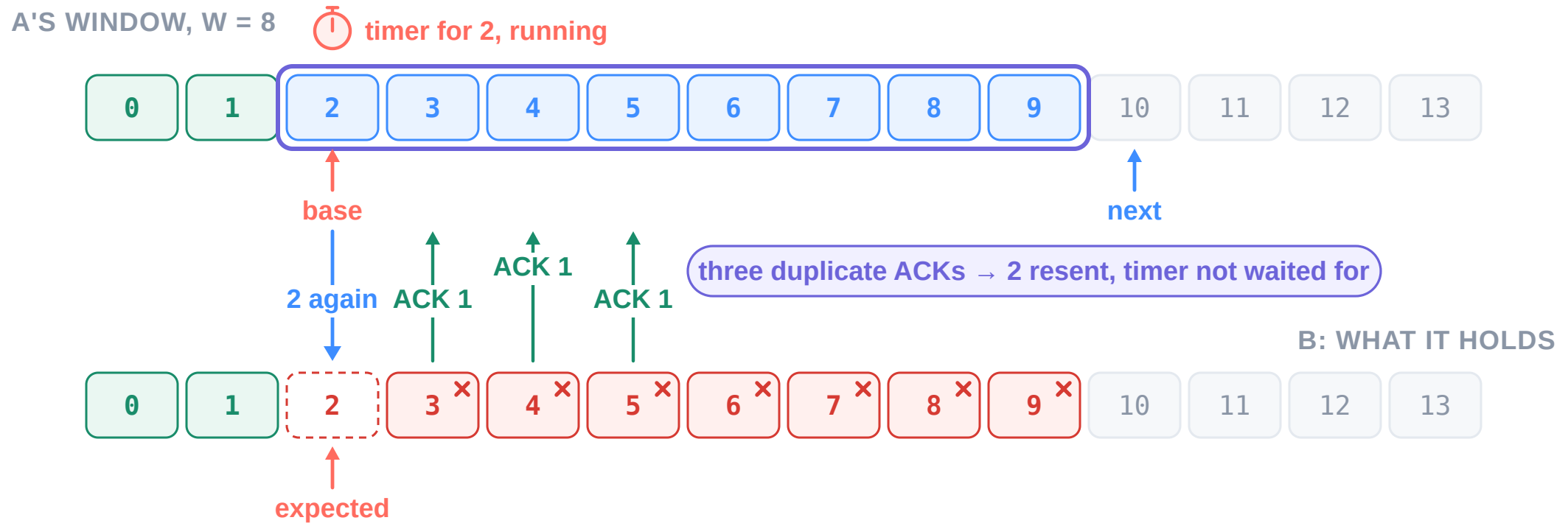
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

Fast retransmit: after three duplicate ACKs, A resends 2



Three duplicate ACK 1s in a row: A treats 2 as lost and resends it at once, without waiting for the timer. Fast retransmit saves the idle timeout. What it cannot save, in Go-Back-N, is resending 3 ... 9: B threw them away.

The timeout is gone from the bill. The W resends are not: 3 ... 9 still follow.

1 Pipelining

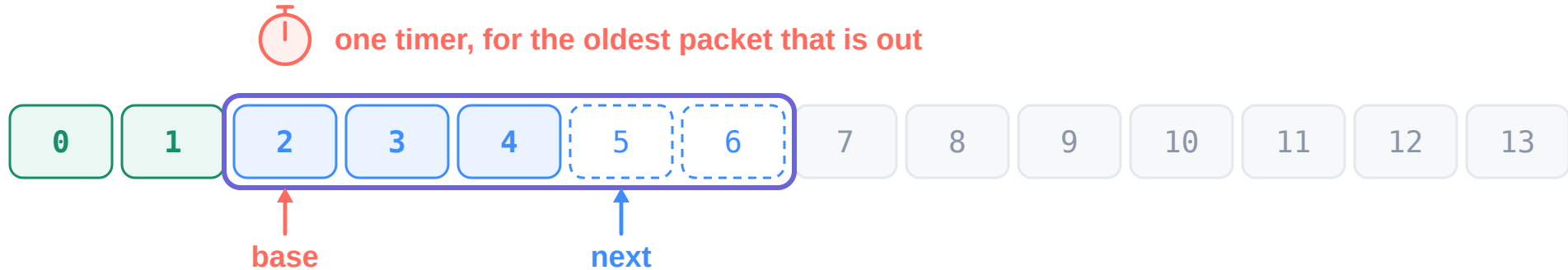
2 Sliding window

3 Go-Back-N

4 Selective repeat

How many timers does A need?

SEEN FROM A: BASE = 2, NEXT = 5, W = 5



ACK n arrives, $n \geq \text{base}$

$\text{base} = n + 1$. If packets are still out ($\text{next} > \text{base}$), restart the timer; otherwise stop it.

the timer expires

resend every packet from base to $\text{next} - 1$ and restart the timer: A goes back to base (Go-Back-N, $N = W$).

ACK n arrives, $n < \text{base}$

nothing new: ignore it. (ACK 1 again, when base is already 2.)

One. It runs for base; when it expires, A resends everything from base on.

1 Pipelining

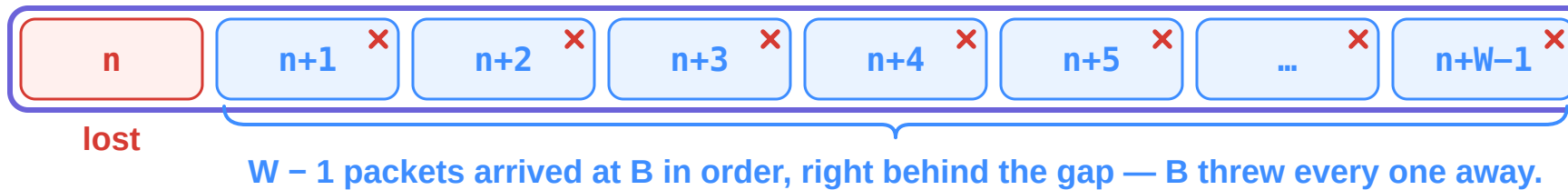
2 Sliding window

3 Go-Back-N

4 Selective repeat

What does one lost packet cost in Go-Back-N?

ONE WINDOW OF W PACKETS IN FLIGHT; THE FIRST ONE, n , IS LOST



W

retransmissions

for a single lost packet: A goes back to n and sends the window again

$W - 1$

good packets wasted

each one crossed the link once, was discarded, and crosses it again

1

timeout of idle time

from the loss to the expiry the link carries only ACK $n - 1$, repeated

Running example: $W = 2,500$. One lost 1,500-byte packet costs up to 2,500 resends: 3.75 MB, a full RTT of link time.

The bigger the window, the bigger the bill. And we made the window big on purpose.

One loss = one timeout + W retransmissions, $W - 1$ of them for packets B already had.

1 **Pipelining**

2 **Sliding window**

3 **Go-Back-N**

4 **Selective repeat**

Go-Back-N in three rules



A: one timer, one window

resend base ... next - 1
when the timer expires;
keep W packets



B: in order only

deliver expected, discard
everything else;
no buffer, one number



ACKs are cumulative

ACK n = everything up to n;
a repeat of ACK n means
n + 1 is missing

Simple at B, and safe against lost ACKs. The price is paid at A, on every loss.

The receiver is as simple as in stop-and-wait. The sender resends the whole window.

1 **Pipelining**

2 **Sliding window**

3 **Go-Back-N**

4 Selective repeat

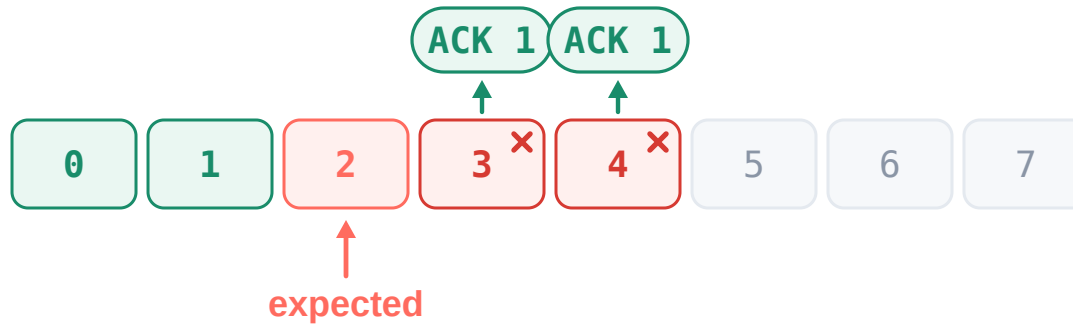
Does A have to resend packets that B already has?

04

Selective repeat

B had 3 and 4 in hand and threw them away. This part lets B keep them, and follows what that changes at both ends.

What if B keeps packets 3 and 4 instead of discarding them?

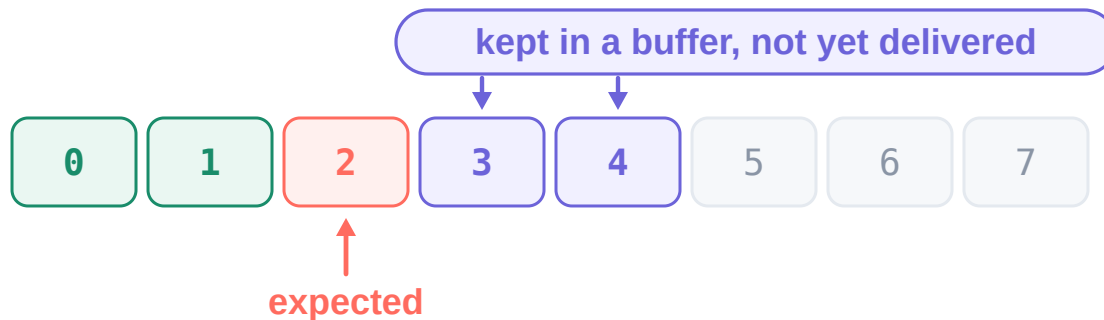


SO FAR · GO-BACK-N

B discards 3 and 4

out of order, so B throws them away
ACK 1 twice: A learns nothing of 3, 4

after the timeout A resends 2, 3 and 4



FROM NOW ON · SELECTIVE REPEAT

B keeps 3 and 4

out of order, so B holds them back
B has 0, 1, 3, 4: only 2 is missing

only 2 needs to cross the link again

The same packets reach B either way. What B does with them decides how much A must send again.

Discard, and A resends the whole window. Keep, and only the lost packet is missing.

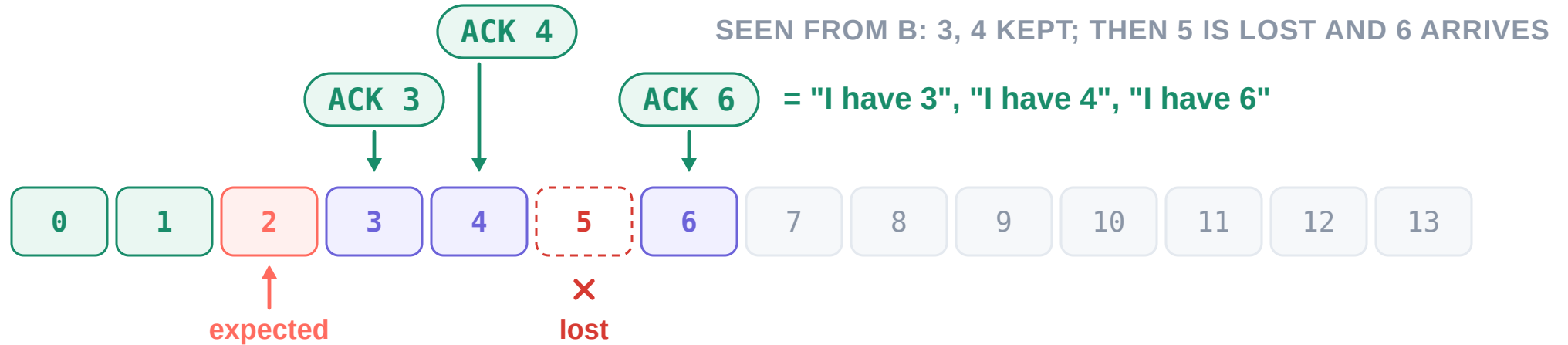
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

What should B tell A now?



individual acknowledgement: ACK n means packet n has arrived, and says nothing about the others

B acknowledges every packet it accepts, in order or not

📄 **Go-Back-N: ACK 1, ACK 1, ACK 1, ACK 1**

0 and 1 arrived; nothing known about 3, 4, 6

✓ **Selective repeat: ACK 1, ACK 3, ACK 4, ACK 6**

exactly what B holds: only 2 and 5 are missing

ACK 3, ACK 4, ACK 6: each ACK names one packet. A learns exactly what B holds.

1 **Pipelining**

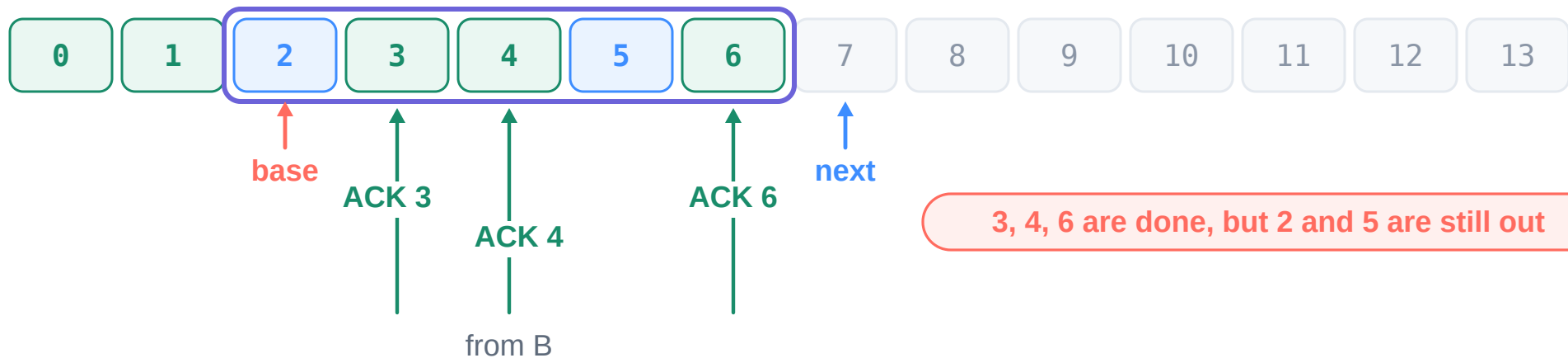
2 **Sliding window**

3 **Go-Back-N**

4 **Selective repeat**

And at A? What does its window look like now?

SEEN FROM A: 0, 1 ACKNOWLEDGED; 2 ... 6 OUT, THE WINDOW IS FULL; $W = 5$



the window does not slide

base moves only when base itself is acknowledged.
2 is not, so the window stays at 2 ... 6, and it is full.

a window with holes in it

done packets sit behind missing ones. A must mark
every slot done or not: base and next are not enough.

3, 4, 6 are done; 2 and 5 are not. The window has holes in it and cannot slide.

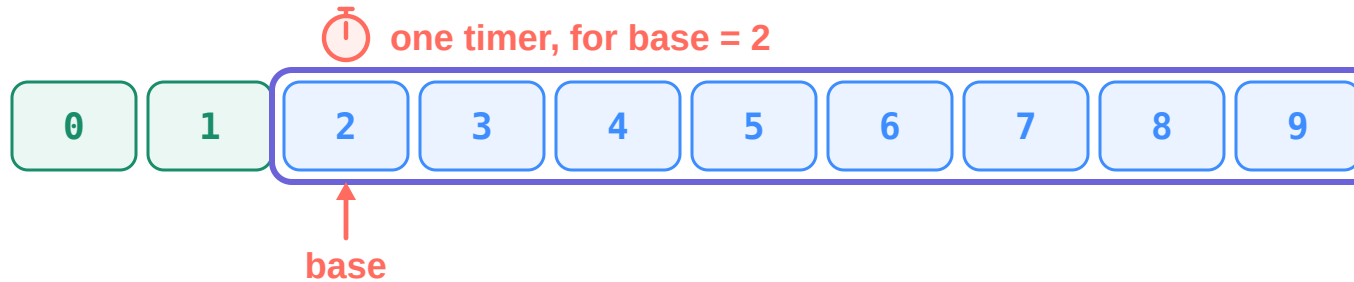
1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

How many timers does A need now?

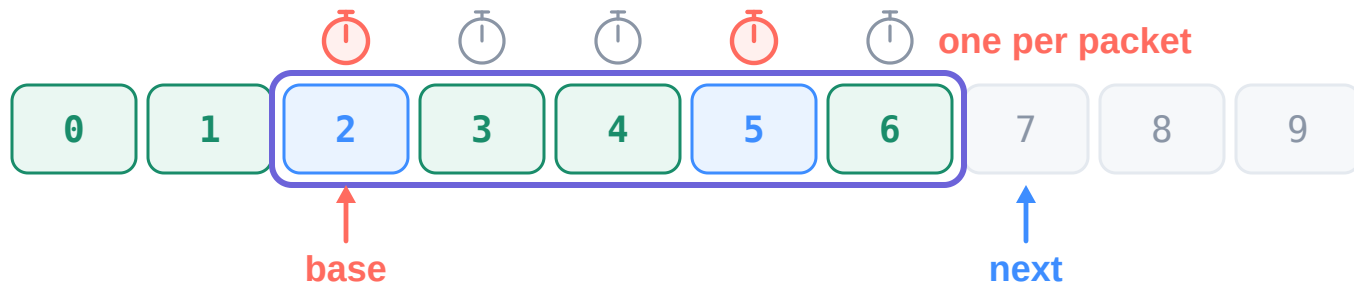


GO-BACK-N · $W = 8$

One timer, for base

it expires → A resends the whole window, 2 ... 9 together
the ACKs are cumulative

resends are not independent



SELECTIVE REPEAT · $W = 5$

One timer per packet

each packet is acknowledged on its own, so it is timed
and resent on its own

resends are independent

One per packet out. Each packet is acknowledged alone, so it is resent alone.

1 Pipelining

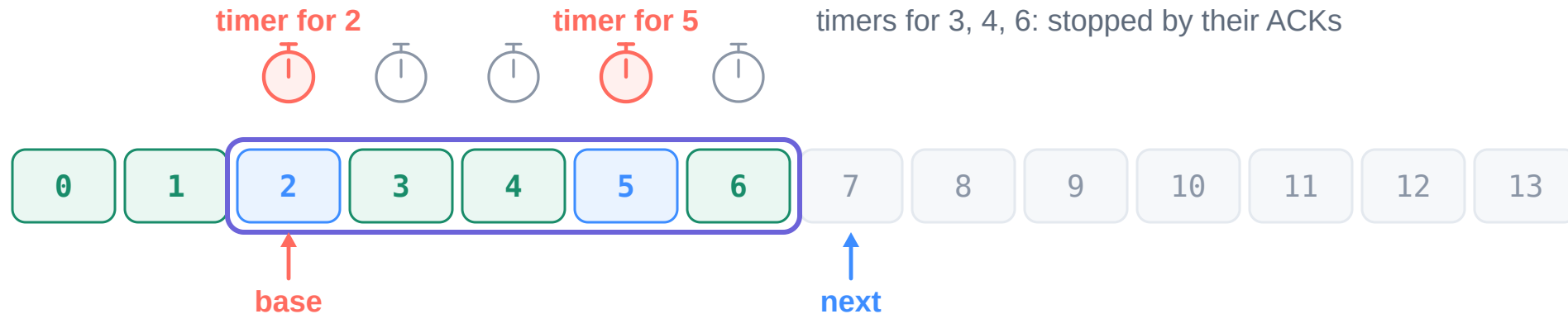
2 Sliding window

3 Go-Back-N

4 Selective repeat

What does A do when a timer expires?

SEEN FROM A: 0, 1 ACKNOWLEDGED; 2, 5 OUT; 3, 4, 6 ACKNOWLEDGED; $W = 5$



resent, each on its own timer

ACK n arrives: mark n done, stop its timer. timer n expires: resend packet n only

Each packet is acknowledged, timed and resent on its own. Nothing else in the window moves.

Resend that packet only. 2 and 5 go again on their own timers; 3, 4, 6 are safe.

1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat

When does A's window slide?

SEEN FROM A: 2 AND 5 ARE STILL OUT; ACK 2 ARRIVES



ACK 2

ACK base arrives: base moves to the oldest packet that is not yet done

to 5, not past it: 5 is still out. When ACK 5 comes, base will jump past 6 to 7.

Only when base is done. Then it jumps to the oldest packet not done: here 2 → 5.

1 Pipelining

2 Sliding window

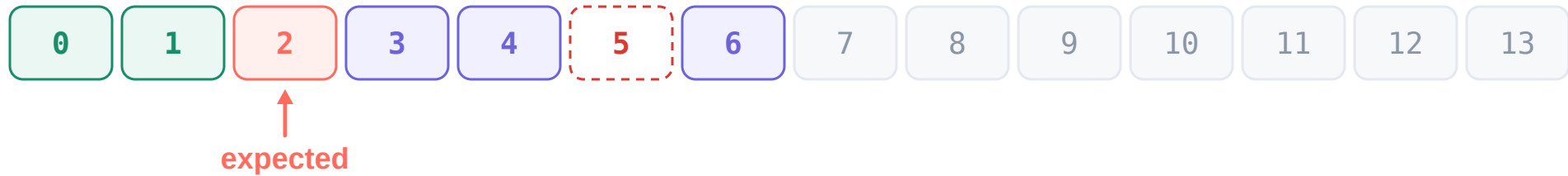
3 Go-Back-N

4 Selective repeat

What happens at B when packet 2 finally arrives?

SEEN FROM B: 3, 4, 6 BUFFERED, 5 LOST; 2 ARRIVES AT LAST

to the application, in order



from the link

packet expected arrives: deliver it and every buffered packet after it, in order; move expected past them
B's window slides by three at once, and stops at the next hole. Its buffer holds at most W packets.

B delivers 2, 3 and 4 together and moves expected to 5. 6 waits in the buffer.

1 Pipelining

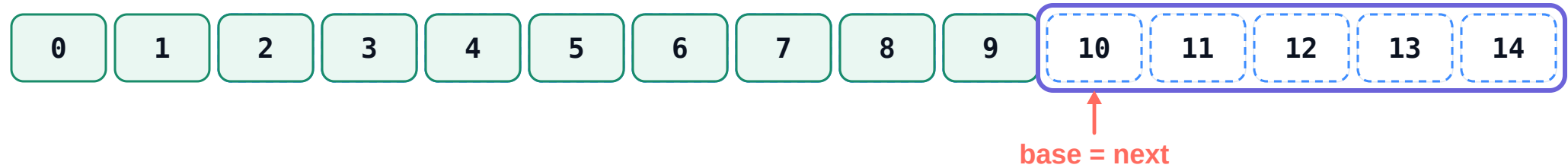
2 Sliding window

3 Go-Back-N

4 Selective repeat

Selective repeat: the whole run, one step at a time

A'S WINDOW, $W = 5$



two losses, two resends

B: WHAT IT HOLDS



Step 5 · The timer for 5 expires: A resends 5. B delivers 5 ... 9 at once and sends ACK 5.

base jumps past 6 ... 9 to 10. Two packets were lost, two were resent; 3, 4, 6, 7, 8, 9 crossed the link once.

Two losses, two resends. Everything B kept was never sent twice; the windows slid over it.

1 **Pipelining**

2 **Sliding window**

3 **Go-Back-N**

4 **Selective repeat**

Three designs in one table

	in flight	sender keeps	receiver keeps	ACK	timers
stop-and-wait	1	1 packet	nothing	per packet	1
Go-Back-N	W	W packets, base, next	nothing	cumulative	1
selective repeat	W	W packets, a done mark each	up to W packets	per packet	1 per packet

Same losses, same window: Go-Back-N resends the whole window, selective repeat only the lost packets.

The difference is paid for with a receive buffer, a done mark per slot, and a timer per packet.

More state at both ends buys fewer retransmissions.

1 Pipelining

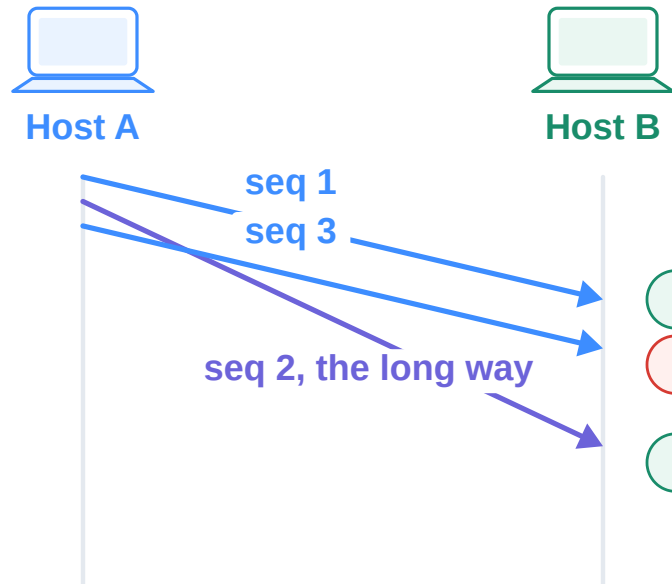
2 Sliding window

3 Go-Back-N

4 Selective repeat

Does the window also handle reordering?

PACKET 3 TAKES A SHORTER ROUTE AND ARRIVES BEFORE PACKET 2



Go-Back-N

3 is discarded and ACK 1 repeated;
A resends 3 after 2. Nothing is lost,
one packet is sent twice.

Selective repeat

3 is buffered and acknowledged;
when 2 arrives, 2 and 3 are delivered
in order. Nothing is sent twice.

⚡ bit flips



🗑 loss



⌚ delay



🔄 reordering



Yes. An early packet is out of order at B: buffered or discarded, not delivered.

1 Pipelining

2 Sliding window

3 Go-Back-N

4 Selective repeat